

Received 5 January 2026; revised 13 May 2026; accepted 21 May 2026; date of publication 25 May 2026;
date of current version 26 June 2026.

Digital Object Identifier 10.1109/TQE.2026.3696707

ZAP: Zoned Architecture and Performant Compiler for Field-Programmable Atom Array

CHEN HUANG^{1,2,6}, XI ZHAO^{1,2,7}, HONGZE XU^{1,2}, WEI-FENG ZHUANG^{1,2},
MENG-JUN HU^{1,2}, DONG E. LIU^{1,2,3,4,5}, AND JINGBO WANG^{1,2}

¹Beijing Academy of Quantum Information Sciences, Beijing 100193, China

²Beijing Key Laboratory of Fault-Tolerant Quantum Computing, Beijing 100193, China

³Frontier Science Center for Quantum Information, Beijing 100184, China

⁴CAS Key Laboratory of Quantum Information, University of Science and Technology of China, Hefei 230026, China

⁵State Key Laboratory of Low Dimensional Quantum Physics, Department of Physics, Tsinghua University, Beijing 100084, China

⁶Department of Computer Science and Engineering, The Chinese University of Hong Kong, Shatin, Hong Kong SAR, China

⁷Hefei National Laboratory, Hefei 230088, China

Corresponding authors: Meng-Jun Hu; Dong E. Liu; Jingbo Wang (email: humj@baqis.ac.cn; dongeliu@mail.tsinghua.edu.cn; wangjb@baqis.ac.cn)

(Chen Huang, Xi Zhao, and Hongze Xu contributed equally to this work.).

The work of Meng-Jun Hu and Jingbo Wang was supported by the National Key Research and Development Program of China under Grant 2025YFE0217200. The work of Dong E. Liu was supported in part by Beijing Natural Science Foundation under Grant Z220002, in part by the National Natural Science Foundation of China under Grant 92365111, in part by the Innovation Program for Quantum Science and Technology under Grant 2021ZD0302400, and in part by the National Natural Science Foundation of China under Grant 92365206.

Data and code and ZAP are open source available online at <https://github.com/BAQIS-Quantum/neutral-atom-compilation>.

ABSTRACT The scalability of neutral-atom quantum computing is increasingly limited by a compiler–architecture challenge: logical circuits must be mapped onto dynamically reconfigurable atom arrays while controlling crosstalk, transport overhead, and hardware constraints. To address this problem, we present Zoned Architecture and Performant (ZAP), a co-designed zoned architecture and deterministic compiler for field-programmable atom arrays. ZAP partitions the array into storage and entanglement zones and combines hardware-aware as-soon-as-possible (ASAP)-separate scheduling, look-ahead placement, and conflict-aware routing in a single-pass compilation flow, thereby avoiding the repeated global search used in prior approaches. Evaluated on structured quantum benchmarks and random three-regular circuits, ZAP consistently delivers multiorder-of-magnitude compilation speedups while maintaining competitive or superior execution quality. Relative to ZAC and PowerMove, ZAP typically reduces compilation time from tens of seconds to below 0.1 s and achieves speedups exceeding 1000×; relative to Enola, the speedup exceeds 10 000× on the evaluated suite. ZAP’s fidelity gains are most pronounced on structured workloads with irregular connectivity and nonuniform qubit reuse, where its scheduling and placement decisions more effectively suppress crosstalk and limit transport-related loss, while on random circuits it remains competitive and preserves the same scalability advantage. These results show that hardware-structured, noniterative compilation provides a practical path toward fast, scalable, and noise-aware neutral-atom quantum computing.

INDEX TERMS Neutral atoms, quantum compiler, quantum computing.

I. INTRODUCTION

In recent years, quantum computing has seen significant advancements in both software and hardware [1], [2], [3], [4], [5]. These improvements have expanded the computational capabilities of quantum systems and opened up potential applications across various fields [6], [7], [8], [9], [10], [11]. Driven by the advances in laser technology, neutral

atom arrays have recently emerged as a leading platform for scalable quantum computing [12], [13], [14], [15], [16], demonstrating systems with thousands of physical qubits [17] and high-fidelity gate operations [18]. Unlike architectures with static qubits, such as superconducting circuits, neutral atom systems feature dynamic qubit connectivity: atoms can be physically moved and

repositioned in real-time using optical tweezers [19]. This architectural capability eliminates the need for costly SWAP gate chains but introduces a formidable hardware–software co-design challenge [20]: how to efficiently compile a quantum circuit to orchestrate thousands of mobile atoms.

The core of this challenge lies in taming the immense search space of potential atom placements and movements. Existing compilers for this paradigm, such as Enola [21] and ZAC [22], attack this problem with slow, iterative, search-based algorithms, while PowerMove [23] is noniterative in placement but remains iterative in routing. While functional for small circuits, these approaches fundamentally fail to scale. Compilation times explode from minutes to hours for modestly sized problems, creating an untenable bottleneck for future fault-tolerant systems and severing the rapid feedback loop essential for quantum algorithm research. This compilation overhead, coupled with physical constraints like atom transport time and crosstalk, forms a critical barrier to unlocking the full potential of neutral atom quantum computers.

To dismantle this barrier, we introduce Zoned Architecture and Performant (ZAP), a performant compiler framework tailored for zoned neutral atom architectures. ZAP’s central insight is to impose structure on the hardware to simplify the compilation task. We spatially partition the atom array into two distinct regions: a compact storage zone for idle-qubits and an optimized entanglement zone where all two-qubit gates are executed. This physical separation of concerns is the key to our performance, significantly decreasing crosstalk. ZAP employs a deterministic, noniterative placement algorithm that positions active qubits in the entanglement zone with minimal movement. This is combined with a hardware-aware scheduling policy: within a standard as-soon-as-possible (ASAP) framework, ZAP separates the scheduling of two-qubit and single-qubit gates to reduce unnecessary movement in zoned architectures. Coupled with look-ahead placement and routing, this policy balances parallelism against crosstalk and atom transport overhead. The result is a system where the hardware architecture and compiler work in concert, transforming a computationally intractable mapping problem into a deterministic and highly efficient process.

Our approach yields transformative improvements in both compilation speed and execution quality. By eliminating iterative placement, ZAP achieves compilation speedups of multiple orders of magnitude compared with prior compilers. On a 100-qubit circuit, compilation completes in under 0.1 s on a laptop, whereas other compilers require minutes or even hours. These speedups are accompanied by fidelity gains driven by reduced atom movement and mitigated crosstalk.

Our main contributions are summarized as follows.

- 1) *Fast and Deterministic Placement for Zoned Architectures*: We develop a deterministic, single-pass placement algorithm tailored to the topology of zoned architectures. Beyond minimizing movement distance, the

placement heuristic incorporates a lightweight routing-aware preference that favors movement patterns with fewer anticipated acousto-optic deflector (AOD) conflicts, thereby removing the critical iterative-search bottleneck found in prior work.

- 2) *Hardware-Aware Scheduling and Routing for Zoned Architectures*: Within a standard ASAP framework, we adopt a separate scheduling policy for two-qubit and single-qubit gates and couple it with look-ahead placement and routing decisions. Unlike prior zoned compilers that either always retain qubits in the entanglement zone or always return them after gate execution, ZAP introduces a hybrid idle-qubit management strategy that dynamically balances crosstalk exposure against transport overhead.
- 3) *Strong Empirical Performance*: We demonstrate that ZAP’s co-design reduces compilation times from minutes to seconds or less while simultaneously improving execution quality.

The rest of this article is organized as follows. Section II reviews neutral atom quantum computing. Section III presents our proposed zoned architecture and the compilation challenges it addresses. Section IV introduces our fidelity analysis. Section V describes the ZAP compiler framework, including its scheduling, placement, and routing algorithms. Sections VI and VII present the experimental setup and evaluation results. Section VIII discusses related work. Finally, Section IX concludes this article.

II. BACKGROUND

A. QUANTUM COMPUTING

Quantum computing operates by manipulating quantum bits (qubits), the fundamental units of quantum information. Unlike a classical bit, a qubit can exist in a superposition of two basis states, $|0\rangle$ and $|1\rangle$, expressed as

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad (1)$$

where α and β are complex probability amplitudes satisfying the normalization condition $|\alpha|^2 + |\beta|^2 = 1$.

Quantum algorithms are composed of a series of quantum gate operations. Single-qubit gates, such as the Hadamard gate, are used to create superposition, while two-qubit gates, such as the controlled-NOT (CNOT) gate, are used to establish entanglement between qubits. According to the Solovay–Kitaev theorem [24], any complex quantum operation can be approximated to arbitrary precision using a finite set of single- and two-qubit gates. This “universal gate set” forms the foundation for constructing all quantum algorithms.

B. NEUTRAL ATOM QUANTUM PLATFORM

In neutral atom quantum computing, qubits are typically represented by two stable internal hyperfine ground states of an atom, corresponding to the logical states $|0\rangle$ and $|1\rangle$. This approach leverages quantum mechanical principles to enable precise control of qubit operations through laser pulses.

Single-qubit gates are achieved by inducing transitions between the hyperfine states using laser pulses, where the pulse duration and phase determine the operation parameters. These operations can be described by unitary matrices of the form

$$U_{\text{single}} = \begin{pmatrix} \cos \theta & -ie^{i\phi} \sin \theta \\ -ie^{-i\phi} \sin \theta & \cos \theta \end{pmatrix} \quad (2)$$

where θ and ϕ are controlled through laser modulation. High-fidelity single-qubit operations have been demonstrated on neutral atom platforms, ensuring their reliability for quantum computation.

A key challenge in neutral atom quantum computing is the implementation of high-fidelity two-qubit gates, which rely on the Rydberg blockade effect. When an atom is excited to a high-energy Rydberg state, it exerts a strong dipole–dipole interaction with surrounding atoms, which “blocks” neighboring atoms within a certain radius from being excited to the same Rydberg state. This conditional mechanism is the physical basis for realizing controlled two-qubit gates, such as the controlled-Z (CZ) gate, which can be represented by the matrix

$$U_{\text{CZ}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}. \quad (3)$$

The CZ gate can be transformed into a CNOT gate by applying Hadamard gates to the target qubit before and after the CZ gate: $U_{\text{CNOT}} = (I \otimes H) \cdot U_{\text{CZ}} \cdot (I \otimes H)$, where I is the identity operator on the control qubit. This transformation allows flexible implementation of two-qubit operations and is widely employed in neutral atom platforms. Therefore, high-fidelity single-qubit gates and CZ gates constitute the fundamental gate operations on neutral atom platforms [16], [18].

III. HARDWARE CONSTRAINTS

The architectural design of a neutral atom quantum computer is fundamentally shaped by the capabilities and limitations of its underlying control hardware. This section details these physical constraints and shows how they motivate our zoned architecture, which in turn defines the core challenges for the compiler.

A. SPATIAL LIGHT MODULATOR (SLM) AND AOD

Qubits in neutral atom quantum computing platforms are encoded in individual atoms trapped by optical tweezers. The spatial positioning of these atoms is controlled by manipulating the optical tweezers. Two key optical devices facilitate this manipulation: the SLM and the AOD. While both can trap atoms, they serve distinct and complementary roles [17], [19].

The SLM generates a fixed 2-D lattice of tightly focused optical traps, where each site can stably hold a single atom.

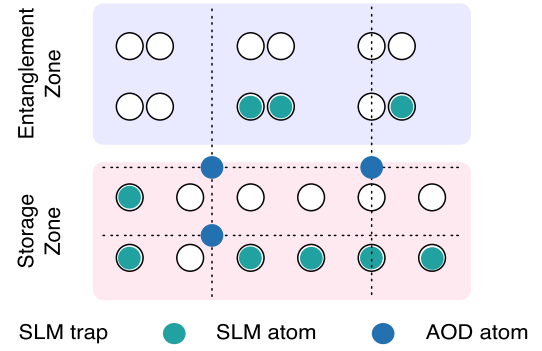


FIGURE 1. Zoned architecture for neutral atom quantum computing. The qubit array is divided into a storage zone and an entanglement zone. An SLM generates static optical traps (white circles) for qubit sites, while AODs create mobile tweezers (dashed lines) to transport selected atoms (blue circles) between sites.

These sites form the static qubit array, with deep potential wells to minimize atom loss and thermal motion during computation.

In contrast, the AOD provides a dynamic, reconfigurable tweezer array with adjustable spacing along orthogonal directions. Its mobility is governed by structural constraints imposed by the multitone frequency control mechanism: atoms can move only within their respective rows or columns, and each row or column must be displaced as a single unit within a defined movement zone. This gives rise to the equidistant, or grid, constraint: for any two atoms moved simultaneously, the projections of their trajectories onto the row and column axes must remain distinct at all times to avoid hardware conflicts. Consequently, atomic trajectories are not independent; the motion of one qubit is inherently coupled to the positions and paths of others sharing the same AOD line. This coupling turns routing into a coordinated scheduling problem and necessitates specialized protocols for collision-free parallel transport. This limited but rapid mobility is exploited to shuttle atoms between different functional zones of the processor.

In our architecture, as shown in Fig. 1, the SLM-generated lattice provides the qubit sites for two distinct regions: a lower storage zone and an upper entanglement zone. To perform a two-qubit gate, the AODs transport selected qubits from the storage zone into the entanglement zone. Within this zone, any pair of atoms brought within the Rydberg blockade radius can execute a CZ gate via laser excitation. This ability to move qubits on demand for targeted entanglement enables universal quantum computation and is consistent with recent large-scale neutral-atom hardware demonstrations based on coherent transport and zone-based operation [16], [17], [19].

B. ZONED ARCHITECTURE

The zoned architecture is a key design paradigm in neutral atom platforms that spatially segregates quantum operations to enhance fidelity and scalability. This approach is primarily motivated by the challenges of laser control for two-qubit gates. While single-qubit gates use individually addressable

lasers, implementing two-qubit gates via Rydberg blockade is more robustly achieved with a global laser field that illuminates an entire region [25], [26]. Such a field, however, would cause significant crosstalk by affecting idle qubits in its path. The zoned architecture solves this by partitioning the qubit array into dedicated regions, primarily a storage zone and an entanglement zone, as shown in Fig. 1. This physically separates active qubits from idle ones, mitigating crosstalk and preserving fidelity.

The storage zone functions as a high-coherence, stable environment where qubits can be held idle or have single-qubit gates applied. The qubit positions are defined by a static SLM lattice with spacing chosen to reduce unwanted interactions and crosstalk between inactive qubits. This separation helps preserve coherence during circuit execution.

The entanglement zone is the active region where high-fidelity two-qubit gates are performed (single-qubit gates can also be applied here). This zone is precisely targeted by the Rydberg laser system needed to induce the blockade effect for CZ gates. The SLM trap sites in this region are arranged into distinct pairs. Within each pair, the two sites are separated by a distance smaller than the Rydberg blockade radius, allowing their corresponding atoms to become entangled. Conversely, the distance between different pairs is kept larger than the blockade radius to prevent unwanted interactions during concurrent gate operations.

The critical link between these zones is the dynamic atom transport enabled by AODs. Qubits are shuttled from the storage zone to specific locations in the entanglement zone to perform a gate, and then moved back or rearranged for subsequent operations. This physical separation offers significant advantages. It isolates idle qubits in the storage zone from the control fields and potential stray light associated with the entanglement zone, thereby minimizing crosstalk and preserving qubit coherence. Furthermore, it facilitates a more efficient workflow, allowing for the preloading and rearrangement of qubits for upcoming gates while other gates are currently executing, which can improve overall parallelism and reduce circuit execution time. These capabilities are increasingly supported by recent large-scale tweezer-array and architecture-level advances [17], [27].

C. COMPILATION CHALLENGES IN NEUTRAL ATOM PLATFORMS

Compilation for neutral atom platforms is governed by a key physical constraint: the Rydberg blockade effect, which causes atoms to interact as qubits only when they are in close proximity. Consequently, a quantum circuit's logical qubits must be mapped to a physical arrangement of atoms that allows for these interactions. The compiler's primary objective is to orchestrate the necessary atomic movements to perform gates efficiently, minimizing execution time while improving fidelity.

A significant challenge in this platform is maximizing parallelism in gate execution. While single-qubit gates can be applied independently, scheduling two-qubit gates is more

complex due to the constraints imposed by the Rydberg blockade [28], [29], [30]. In principle, neutral atom architectures support concurrent two-qubit operations, provided that active qubit pairs are sufficiently spaced such that their blockade regions do not overlap. Maintaining independent Rydberg blockades requires that, during a two-qubit gate, no other atoms within the blockade radius are inadvertently excited to Rydberg states. This is ensured through precise laser targeting and adequate spatial separation between gate pairs. Violations of this condition can lead to undesired interactions and reduced gate fidelity. Consequently, compilation algorithms must carefully coordinate atomic placement and gate timing to maximize parallelism while preserving fidelity. This spatial constraint makes the efficient execution of large-scale circuits particularly challenging, as reducing circuit depth must be balanced against the need to prevent blockade interference.

A practical strategy for managing gate dependencies and optimizing parallelism in quantum circuits is to represent the circuit as a directed acyclic graph (DAG), where each node corresponds to a quantum gate, and edges indicate execution constraints. This representation helps identify independent operations that can be executed in parallel, reducing the critical path length of the circuit. In neutral atom platforms, where qubit connectivity is dynamically reconfigurable, DAG-based analysis provides a structured way to plan atomic movements and optimize spatial arrangements. By mapping qubit interactions onto the physical layout, the compiler can schedule movements efficiently, minimizing unnecessary delays and mitigating coherence loss.

Among these challenges, this work focuses explicitly on reducing atom-movement time without compromising gate fidelity. Unlike in superconducting platforms, where some speed-fidelity tradeoffs can be managed through pulse shaping or error mitigation, neutral atom systems face stricter movement-related constraints. Atomic motion introduces decoherence and heating, while prolonged interaction times increase susceptibility to noise and errors. Therefore, minimizing movement time while preserving gate fidelity is not merely an optimization goal but a fundamental requirement for maintaining overall circuit performance. Our compilation strategy employs a DAG-based scheduling approach, which we term the ASAP scheduling strategy, combined with spatial optimization. Notably, our method avoids iterative refinement in the placement process, substantially reducing compilation time. By integrating these techniques, we enhance the efficiency and reliability of neutral atom quantum circuit execution.

IV. FIDELITY ANALYSIS

Multiple factors influence the fidelity of quantum circuit execution on neutral atom platforms. In addition to the intrinsic fidelities of single- and two-qubit gates, the finite qubit lifetime constrains gate execution times, impacting overall fidelity. Furthermore, atom transport between SLM and AOD traps introduces heating effects and time delays,

TABLE 1. Key Experimental Parameters Used in This Work

Operation	Notation	Duration / Constraint	Fidelity
Single-qubit gate	f_1	$52 \mu\text{s}$	0.9997
Two-qubit gate	f_2	$0.36 \mu\text{s}$	0.995
Crosstalk	f_{xtalk}	$0.36 \mu\text{s}$	0.9975
Atom transfer	f_{tr}	$15 \mu\text{s}$	0.999
Atom movement	-	$a < 2.75 \mu\text{m} \mu\text{s}^{-2}$	~ 1

The listed gate durations and operation fidelities are based on the values reported in [16], while atom movement is characterized by the acceleration constraint used in our simulator.

further degrading performance. However, the direct fidelity loss from the transport process itself can often be neglected. Experimental work has shown that for sufficiently controlled transport speeds, the process introduces negligible intrinsic error [19]. Therefore, we assume the primary penalty of atom movement is the time it adds to the circuit's duration, which contributes to decoherence, rather than a direct loss of fidelity from the transport itself. Under these assumptions, the overall circuit fidelity can be estimated using the following expression:

$$f = (f_1)^{g_1} \cdot (f_2)^{g_2} \cdot (f_{\text{xtalk}})^{N_{\text{xtalk}}} \cdot (f_{\text{tr}})^{N_{\text{tr}}} \prod_{q \in \mathcal{Q}} \exp\left(-\frac{t_q}{T_2}\right). \quad (4)$$

Here, f_1 and f_2 denote the fidelities of single- and two-qubit gates, respectively, while g_1 and g_2 represent their counts in the circuit. Executing two-qubit gates requires a global light field to excite atoms to the Rydberg state. However, this field can inadvertently affect nontargeted atoms within the entanglement zone, introducing crosstalk noise. The term f_{xtalk} accounts for the fidelity loss due to this effect, with N_{xtalk} representing the number of such occurrences. Furthermore, f_{tr} accounts for the fidelity of atom transfer operations between the SLM and AOD traps and N_{tr} represents the number of transfers occurring during circuit execution. Finally, the impact of decoherence is captured by the product term $\prod_{q \in \mathcal{Q}} \exp(-t_q/T_2)$ over the set of all qubits $\mathcal{Q}$. For each qubit q , the fidelity is reduced based on its idle time t_q and the coherence time T_2 . The idle time is the total circuit duration minus the time the qubit is actively occupied (gate execution, transport, etc). All parameters for this model, listed in Table 1, are based on experimental demonstrations, with the qubit coherence time T_2 set to 1.5 s [19].

For atom movement, we assume near-unit intrinsic fidelity under sufficiently smooth transport, so its primary effect enters through additional runtime rather than through a separate direct error channel. Accordingly, the "Atom movement" row in Table 1 reports the acceleration constraint used in our simulator rather than a fixed duration. For a displacement of length d , assuming a symmetric acceleration–deceleration profile with maximum acceleration a , we use the movement-time model

$$t_{\text{mv}}(d) = 2\sqrt{\frac{d}{a}}. \quad (5)$$

This model is used consistently for all compared compilers and corresponds to an acceleration-limited transport profile

under the bound in Table 1. When a logical movement must be decomposed into multiple routing substages, or when a short parking shift is inserted to resolve AOD conflicts, the total movement overhead is computed as the sum of the durations of all constituent movement steps.

For a given quantum circuit, the gate counts g_1 and g_2 are predetermined by the algorithm's structure. The corresponding gate fidelities, f_1 and f_2 , are primarily limited by the quality of the local laser control and intrinsic atomic properties. Consequently, the gate fidelity term in (4), $(f_1)^{g_1} \cdot (f_2)^{g_2}$, offers limited room for improvement through compilation alone.

Our compilation strategy, therefore, targets the remaining dynamic factors that are highly dependent on the circuit's spatio-temporal layout. The primary optimization goals are to

- 1) minimize crosstalk events N_{xtalk} by optimizing qubit placement and gate scheduling;
- 2) reduce the number of atom transfers N_{tr} to limit transport-related time overhead and errors;
- 3) shorten the total circuit execution time, which directly reduces each qubit's idle time t_q and mitigates decoherence.

By targeting these factors, a compiler can significantly enhance the overall fidelity of quantum computations on neutral atom platforms.

V. ZAP COMPILER

Building on the distinctive features of neutral-atom quantum platforms and the fidelity analysis above, we propose ZAP compiler for field-programmable atom arrays. ZAP leverages the architectural separation of storage and entanglement zones to enable a deterministic, single-pass compilation process. This approach removes the primary performance bottleneck of prior methods, leading to significant gains in both compilation speed and execution fidelity. The ZAP compilation flow consists of three key steps: scheduling, mapping and placement, and routing, which are described in the following sections.

A. QUBIT SCHEDULING

Before execution, a quantum circuit must be transpiled into a sequence of single-qubit and cz gates. The circuit's performance is often characterized by its depth, defined by the number of sequential stages. Here, a stage denotes a logical scheduling block rather than a guarantee that all contained operations are physically executed simultaneously; operations with incompatible hardware requirements may be temporally ordered within the same stage.

ASAP scheduling is a standard technique for minimizing circuit depth. In zoned neutral-atom architectures, however, different ASAP variants interact quite differently with qubit-movement overhead. We therefore distinguish two execution policies within the same ASAP framework: a joint scheduling strategy (denoted as ASAP-joint), which assigns

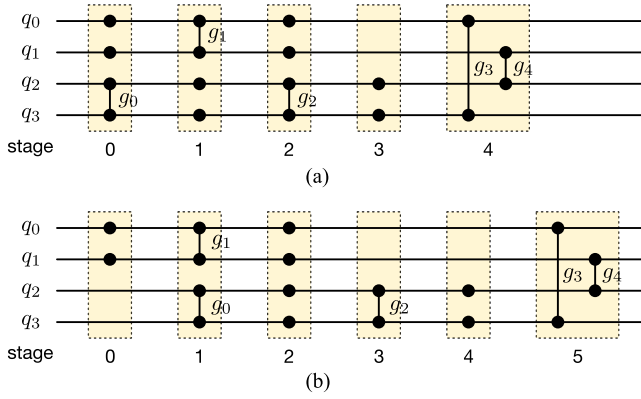


FIGURE 2. (a) ASAP-joint: assigns single- and two-qubit gates to the same logical scheduling blocks to minimize logical depth. (b) ASAP-separate: schedules two-qubit gates first, then fills single-qubit gates into available slots. This strategy may increase stage count but better matches zoned architectures by reducing shuttling overhead, and is adopted in ZAP.

single- and two-qubit gates to the same logical scheduling blocks, and a separate scheduling strategy (denoted as ASAP-separate), which schedules two-qubit gates first and then fills the remaining temporal slots with single-qubit gates.

The ASAP-joint strategy assigns all gate types to the earliest possible logical stages to minimize the total number of stages, as shown in Fig. 2(a). While this approach reduces logical depth, it creates tightly coupled dependencies between consecutive stages and therefore often requires frequent qubit movement. For example, as illustrated in Fig. 3, ASAP-joint introduces frequent movement between consecutive stages (e.g., stages 0–1, 1–2, and 3–4) due to tightly coupled gate dependencies. In contrast, ASAP-separate significantly reduces movement by grouping two-qubit interactions and deferring single-qubit operations.

In contrast, ZAP adopts the ASAP-separate strategy, which first schedules all two-qubit gates and then fills the remaining temporal slots with single-qubit gates [see Fig. 2(b)]. By decoupling the scheduling of gate types, this approach reduces unnecessary qubit movement and simplifies downstream placement and routing. In the same example, qubit movement is required only between stages 0 and 1, and between stages 4 and 5.

Importantly, grouping single- and two-qubit gates within the same stage does not imply simultaneous execution. In our execution model, operations within a stage are temporally ordered to respect hardware constraints. In particular, global Rydberg entangling operations and single-qubit operations that cannot be applied concurrently are serialized within the stage, whereas physically compatible operations acting on different zones can still be executed in parallel.

This design reflects an important tradeoff: although ASAP-separate may increase the number of stages, it significantly reduces qubit shuttling, which is a dominant source of decoherence and operational overhead in zoned

architectures. As a result, it improves overall execution efficiency despite the potentially larger circuit depth.

B. QUBIT MAPPING AND PLACEMENT

ZAP adopts a deterministic, routing-aware placement strategy. Rather than solving a costly global optimization problem, it combines a one-time initial mapping with a lightweight stage-wise refinement that explicitly favors movement patterns likely to be executable in parallel under AOD constraints. This design preserves compilation efficiency while tightly coupling placement to the downstream routing procedure.

Initial Mapping: At the start of compilation, all qubits are assigned to storage sites. We first rank the storage sites according to their proximity to the entanglement zone. For each storage site $u \in \mathcal{S}$, let its average distance to the entanglement sites $\mathcal{E}$ be

$$D(u) = \frac{1}{|\mathcal{E}|} \sum_{e \in \mathcal{E}} \|u - e\| \quad (6)$$

and let its nearest entanglement site be

$$e^*(u) = \arg \min_{e \in \mathcal{E}} \|u - e\|, \quad d_{\min}(u) = \|u - e^*(u)\|. \quad (7)$$

Qubits are prioritized according to their expected importance in the circuit, with earlier stages receiving larger weights

$$w(\ell) = \frac{1}{\ell + 1}, \quad \ell \in \{0, 1, 2, \dots, n_{\text{stage}} - 1\} \quad (8)$$

where ℓ is the stage index, with $\ell = 0$ denoting the first stage, and gate contributions decay as ℓ increases. The total weight for qubit q is then computed as

$$W(q) = \sum_{\ell} \sum_{g \in \mathcal{G}_{\ell}, q \in g} w(\ell) \quad (9)$$

where $\mathcal{G}_{\ell}$ denotes the set of gates in stage ℓ . This weighting scheme emphasizes near-term interactions and gives higher priority to qubits that participate more frequently in earlier stages.

To further improve routing concurrency, the initial mapping incorporates a parallel-awareness term. For each qubit q , we identify the first stage $\ell_{2q}^{\text{first}}(q)$ in which it participates in a two-qubit gate. If this first two-qubit use occurs within a short look-ahead window, then assigning q to a storage site u induces an estimated first movement vector from u to $e^*(u)$. We denote this vector by

$$v(u) = (u_x, e_x^*(u), u_y, e_y^*(u)). \quad (10)$$

For a candidate site u , we count how many already planned vectors are incompatible with $v(u)$ under the same 2-D compatibility rule later enforced by the router. Denoting this count by $c(u)$, we score each candidate site as

$$\text{score}(u) = \lambda_{\text{par}} c(u) + d_{\min}(u) \quad (11)$$

where λ_{par} controls the priority given to future parallel transport. Thus, the initial mapping balances two goals: placing

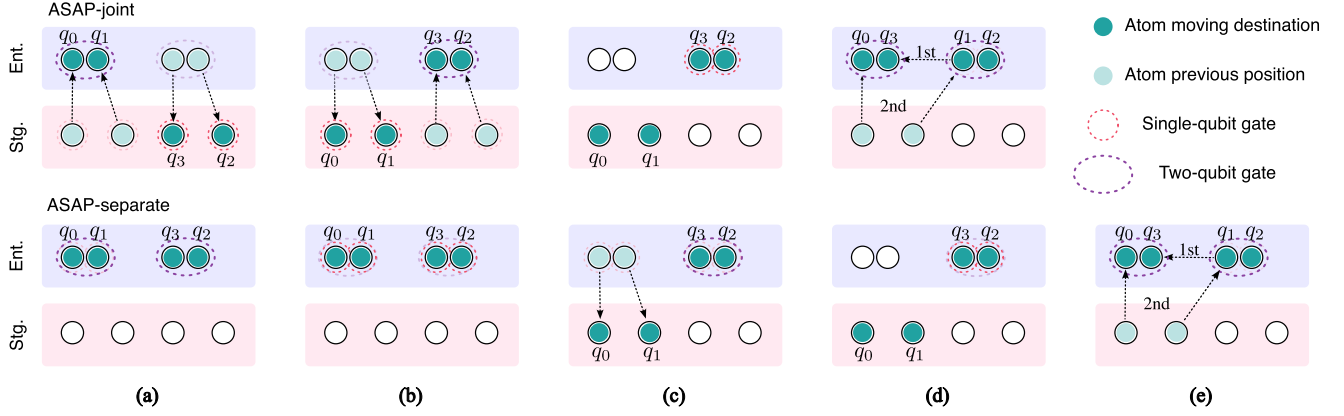


FIGURE 3. Step-by-step illustration of qubit movement across logical stages under the two scheduling strategies in Fig. 2. The top row shows ASAP-joint scheduling, in which tightly coupled dependencies introduce frequent atom movements between consecutive stages. The bottom row shows ASAP-separate scheduling, in which two-qubit gates are grouped and single-qubit gates are deferred, significantly reducing interstage movement. Dashed arrows indicate atom transport, and highlighted regions denote gate-execution zones. Mixed stages in ASAP-joint are logical groupings; under hardware constraints, their single- and two-qubit operations remain temporally ordered. (a) From stage 0 to stage 1. (b) From stage 1 to stage 2. (c) From stage 2 to stage 3. (d) From stage 3 to stage 4. (e) From stage 4 to stage 5.

important qubits near the entanglement zone and making their early movements more likely to be routed concurrently.

Stage-Wise Placement: After the initial mapping, the placer updates the qubit layout stage by stage. For each stage, it first handles qubits that remain in the entanglement zone but are not active in the current two-qubit stage. Unlike prior zoned compilers that typically move idle qubits back to the storage zone after two-qubit operations, ZAP dynamically decides on a per-qubit and per-stage basis whether an idle qubit should remain in the entanglement zone or be returned to storage. This decision is governed by an explicit trade-off between leaving an idle qubit in the entanglement zone, which may expose it to repeated global two-qubit illumination, and moving it back to storage, which incurs atom-transfer and decoherence costs.

For an idle qubit q at stage ℓ , let $\ell_{\text{next}}(q)$ denote the next stage in which q participates in a two-qubit gate. If no such stage exists, we set $\ell_{\text{next}}(q)$ to the end of the circuit. Let $k_{q,\ell}$ denote the number of stages during which q would remain exposed to two-qubit-zone crosstalk if it stays in place. The resulting crosstalk loss is estimated as

$$L_{q,\ell}^{\text{xtalk}} = k_{q,\ell} (-\ln f_{2q,\text{idle}}) \quad (12)$$

where $f_{2q,\text{idle}}$ is the effective fidelity assigned to an idle qubit during a two-qubit stage. If q is moved back to storage, the compiler instead incurs a transfer loss

$$L_{q,\ell}^{\text{tr}} = n_{\text{tr}}(q, \ell) (-\ln f_{\text{tr}}) \quad (13)$$

where $n_{\text{tr}}(q, \ell) = 2$ if q is never used again in a two-qubit gate and $n_{\text{tr}}(q, \ell) = 4$ otherwise. In addition, moving the qubit introduces extra runtime and hence additional decoherence. Let $d_{q,\ell}$ be the distance from its current location to the selected storage site, and let $t_{\text{mv}}(d_{q,\ell})$ defined by (5) denote the movement duration model. The decoherence term

is approximated as

$$L_{q,\ell}^{\text{dec}} \approx \alpha_{\text{idle}} \frac{(N_q - 1) \tilde{t}_{q,\ell}}{T_2} \quad (14)$$

where N_q is the total number of qubits, T_2 is the coherence time, α_{idle} is a calibration weight, and $\tilde{t}_{q,\ell}$ is the estimated additional movement time associated with sending the qubit out and, if needed, bringing it back later. The placement decision is then

$$x_{q,\ell}^* = \begin{cases} 1, & \text{if } L_{q,\ell}^{\text{xtalk}} > L_{q,\ell}^{\text{tr}} + L_{q,\ell}^{\text{dec}} \\ 0, & \text{otherwise} \end{cases} \quad (15)$$

where $x_{q,\ell}^* = 1$ means that q is returned to storage.

After idle-qubit management, the placer assigns target entanglement sites for the active two-qubit gates of the current stage. If one qubit of a gate already occupies one site of a valid entanglement pair, the other qubit is moved to the partner site whenever possible. Otherwise, the placer selects a free entanglement pair. When multiple free entanglement pairs are available, ZAP deterministically selects the pair that minimizes the routing-aware placement score in (17). This lightweight greedy heuristic favors physically shorter movements while simultaneously reducing anticipated AOD conflicts in the subsequent routing stage.

Concretely, if qubit q is moved to site p , the placer forms its movement vector and counts incompatibilities with previously committed vectors in the same stage. For a single move, the score takes the form

$$\text{score}_{\text{single}}(q, p) = \lambda_{\text{par}} \text{conflicts}(q, p) + \|M(q) - p\| \quad (16)$$

and for a two-qubit placement onto an ordered pair (p_0, p_1) the score becomes

$$\begin{aligned} \text{score}_{\text{pair}}(q_0, q_1, p_0, p_1) &= \lambda_{\text{par}} \text{conflicts}(q_0, q_1, p_0, p_1) \\ &\quad + \|M(q_0) - p_0\| + \|M(q_1) - p_1\|. \end{aligned} \quad (17)$$

Algorithm 1: Parallel-Aware Initial Mapping.

Input: Storage sites $\mathcal{S}$, entanglement sites $\mathcal{E}$, qubits $\mathcal{Q}$, full stage schedule Π , look-ahead window L , parallel weight λ_{par}

Output: Initial mapping M_0

```

1: for all  $u \in \mathcal{S}$  do
2:   Compute  $D(u)$  using (6)
3:   Compute  $e^*(u)$  and  $d_{\min}(u)$  using (7)
4: end for
5:  $\mathcal{S}_{\text{sorted}} \leftarrow$  storage sites sorted by increasing  $D(u)$ 
6: for all  $q \in \mathcal{Q}$  do
7:   Compute  $W(q)$  using (9)
8:   Determine  $\ell_{2q}^{\text{first}}(q)$ 
9: end for
10:  $\mathcal{Q}_{\text{sorted}} \leftarrow$  qubits sorted by decreasing  $W(q)$ 
11:  $M_0 \leftarrow \emptyset$ ,  $\mathcal{V}_{\text{plan}} \leftarrow \emptyset$ 
12: for all  $q \in \mathcal{Q}_{\text{sorted}}$  do
13:   for all unassigned site  $u \in \mathcal{S}_{\text{sorted}}$  do
14:     if  $\ell_{2q}^{\text{first}}(q) \neq \perp$  and  $\ell_{2q}^{\text{first}}(q) \leq L$  then
15:       Form vector  $v(u)$  using (10)
16:        $c(u) \leftarrow$  number of vectors in  $\mathcal{V}_{\text{plan}}$  that are
         AOD-incompatible with  $v(u)$  under the
         router's row/column compatibility rule
17:     else
18:        $c(u) \leftarrow 0$ 
19:     end if
20:      $\text{score}(u) \leftarrow \lambda_{\text{par}} c(u) + d_{\min}(u)$ 
21:   end for
22:    $u^* \leftarrow \arg \min_u \text{score}(u)$ 
23:   Assign  $M_0(q) \leftarrow u^*$ 
24:   if  $\ell_{2q}^{\text{first}}(q) \neq \perp$  and  $\ell_{2q}^{\text{first}}(q) \leq L$  then
25:     Add  $v(u^*)$  to  $\mathcal{V}_{\text{plan}}$ 
26:   end if
27: end for
28: return  $M_0$ 

```

Thus, stage-wise placement is not a purely geometric nearest pair assignment. Instead, it incrementally commits qubit movements in a manner that preserves future routing concurrency under AOD constraints.

C. QUBIT ROUTING

Once the target placement for a stage has been determined, the router generates the physical instruction sequence that transforms the current qubit configuration into that target layout. Routing therefore bridges consecutive logical stages: it realizes the qubit movements implied by stage-wise placement and ensures that the required qubits reach valid physical locations before gate execution begins.

Fig. 4 illustrates the stage-level role of routing in ZAP. The circuit is executed as a sequence of stages: single-qubit stages can often be completed directly in the storage zone, whereas two-qubit stages require selected qubits to be transported into the entanglement zone and aligned with valid

Algorithm 2: Routing-Aware Stage-Wise Placement.

Input: Current mapping M , current stage ℓ , stage gates $\mathcal{G}_\ell$, full schedule Π , storage sites $\mathcal{S}$, entanglement pairs $\mathcal{P}$

Output: Target mapping M_{target} for stage ℓ

```

1:  $M_{\text{target}} \leftarrow M$ 
2:  $\mathcal{V}_{\text{plan}} \leftarrow \emptyset$ 
3: Step 1: idle-qubit management
4: for all qubit  $q$  currently in the entanglement zone do
5:   if  $q$  is not active in a two-qubit gate at stage  $\ell$  then
6:     Find  $\ell_{\text{next}}(q)$ 
7:     Compute  $L_{q,\ell}^{\text{talk}}$  using (12)
8:     Select the best candidate storage site
9:     Compute  $L_{q,\ell}^{\text{tr}}$  and  $L_{q,\ell}^{\text{dec}}$  using (13) and (14)
10:    if  $L_{q,\ell}^{\text{talk}} > L_{q,\ell}^{\text{tr}} + L_{q,\ell}^{\text{dec}}$  then
11:      Move  $q$  to storage and commit its
        movement vector
12:    end if
13:  end if
14: end for
15: Step 2: active two-qubit placement
16: for all two-qubit gate  $(q_0, q_1)$  in stage  $\ell$  do
17:   if  $q_0$  and  $q_1$  are already on a valid entanglement
    pair then
18:     Continue to the next gate
19:   else if  $q_0$  already occupies one site of a partially
    filled valid pair then
20:     Move  $q_1$  to the partner site using the lowest
      AOD-aware routing score
21:     Commit all newly selected movement vectors
22:   else if  $q_1$  already occupies one site of a partially
    filled valid pair then
23:     Move  $q_0$  to the partner site using the lowest
      AOD-aware routing score
24:     Commit all newly selected movement vectors
25:   else
26:     Choose a free entanglement pair  $(p_0, p_1) \in \mathcal{P}$ 
27:     Orient  $(p_0, p_1)$  to minimize the AOD-aware
      score in (17)
28:     Assign  $q_0 \rightarrow p_0$  and  $q_1 \rightarrow p_1$ 
29:     Commit all newly selected movement vectors
30:   end if
31: end for
32: return  $M_{\text{target}}$ 

```

interaction pairs. In the example, stage 0 applies single-qubit gates to q_0 and q_1 without movement; stage 1 shuttles q_0 through q_3 into the entanglement zone to execute two two-qubit gates; and the final stage again applies single-qubit gates locally. Routing is therefore not an isolated geometric subroutine, but the mechanism that transforms the current qubit layout into the target layout required by the next stage.

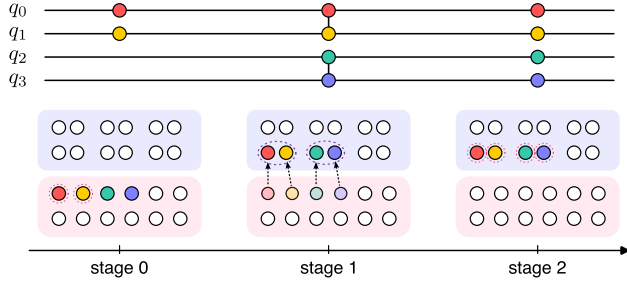


FIGURE 4. Routing of a quantum circuit in ZAP. The circuit is divided into sequential stages, each containing gates that can be executed in parallel. In stage 0, single-qubit gates are applied to q_0 and q_1 in the storage zone. To execute the two-qubit gates in stage 1, qubits q_0 through q_3 are shuttled to the entanglement zone. In stage 2, subsequent single-qubit gates are applied. This dynamic shuttling between zones and dictated by the circuit structure, realizes the complete routing of the quantum circuit.

For a given stage, let M denote the current mapping and let M_{target} denote the target mapping produced by the placer. Each qubit whose source and destination differ induces a movement vector

$$v_q = (x_q^{\text{src}}, x_q^{\text{dst}}, y_q^{\text{src}}, y_q^{\text{dst}}). \quad (18)$$

The full routing task is therefore represented by the set

$$\mathcal{V} = \{v_q \mid M(q) \neq M_{\text{target}}(q)\}. \quad (19)$$

This transport is not executed as one monolithic movement. Instead, the router repeatedly extracts a subset of safe, mutually compatible moves, executes that batch, updates the current mapping, and then continues with the remaining vectors. To avoid immediate target-site contention, the router first identifies the set of safe vectors

$$\mathcal{V}_{\text{safe}} = \{v \in \mathcal{V} \mid \text{target}(v) \notin \mathcal{P}_{\text{start}}\} \quad (20)$$

where $\mathcal{P}_{\text{start}}$ denotes the set of source positions of all unfinished moves. If $\mathcal{V}_{\text{safe}}$ is empty, the router executes a remaining move to break the cycle.

Among the safe vectors, the router determines which moves are compatible under the AOD row/column constraint. Two vectors are compatible only if they preserve row and column ordering and do not request conflicting actuation on the same source or destination line. The router then builds a compatibility graph on $\mathcal{V}_{\text{safe}}$ and selects a greedy maximal independent set as the next routing batch.

Not all conflicts require full serialization. Fig. 5 illustrates a representative routing example in which two atoms start in the same row but must move to different target rows. Rather than serializing the two moves, ZAP first activates both atoms, applies a short parking displacement to the upper atom, and then performs the main movement in parallel. After the atoms reach their destinations, they are deactivated. This example shows how a small intermediate adjustment can convert an otherwise incompatible movement pattern into a parallel one.

More generally, parking operations are inserted whenever direct transport is blocked by AOD selection ambiguity or by geometric interference with static atoms. These auxiliary

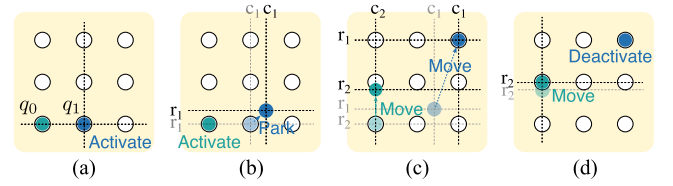


FIGURE 5. Instruction-level example of parking-enabled parallel transport. (a) Two atoms start in the same row and are activated. (b) Short parking displacement is applied to the upper atom to resolve the initial AOD row/column conflict. (c) Two atoms then execute their main movement in parallel toward different target rows. (d) After reaching their destinations, the atoms are deactivated. This example illustrates how ZAP uses a brief parking step to convert an initially incompatible configuration into one that supports concurrent transport.

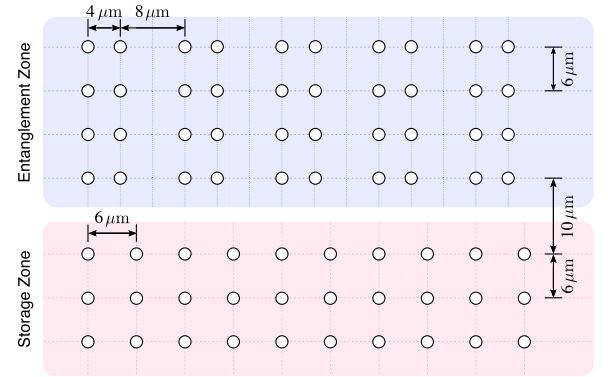


FIGURE 6. Storage zone consists of atoms arranged with a spacing of $6 \mu\text{m}$ in both the x - and y -directions, where qubits are held when not participating in two-qubit gate operations. The entanglement zone features atom pairs with an intrapair spacing of $4 \mu\text{m}$ and a minimum interpair spacing of $6 \mu\text{m}$, facilitating controlled interactions for quantum entanglement operations. For commonly used qubit atoms, the Rydberg blockade radius is typically $5 \mu\text{m}$. For example, the blockade radius is approximately $4.3 \mu\text{m}$ for Rb, which informs the atom arrangement in our simulation.

displacements are explicit parts of the routing instruction stream, and their runtime overhead is included in the physical execution schedule. After each batch is completed, the corresponding vectors are removed from $\mathcal{V}$, the mapping is updated, and the process repeats until all movements are finished. ZAP routing is therefore an iterative safe-batch transport procedure with exact compatibility checking and parking-based local conflict resolution.

VI. EVALUATION SETUP

To comprehensively evaluate the performance of ZAP, we develop a simulation framework implemented in Python 3.12 and executed on a system equipped with an Apple M4 processor and 16 GB of memory. Our evaluation adopts experimentally realistic parameters based on state-of-the-art neutral-atom platforms and compares ZAP against representative baseline compilers.

A. HARDWARE PARAMETER SETTING

For rubidium (Rb) atoms, the Rydberg blockade radius is approximately $4.3 \mu\text{m}$ [16]. Accordingly, as shown in Fig. 6, the atomic spacing in the storage zone was set to $6 \mu\text{m}$ in both

the x - and y -directions. This spacing ensures that atoms in the Rydberg state do not interact with their neighbors and allows each atom to perform single-qubit gates without interference. In the entanglement zone, the spacing between atoms within each qubit pair used for two-qubit gates along the x -direction was set to $4\text{ }\mu\text{m}$, while a minimum separation of $6\text{ }\mu\text{m}$ was maintained between qubit pairs at different sites. This configuration enables parallel entangling operations because each active pair lies within the Rydberg blockade radius while different pairs remain sufficiently separated. To prevent interference between the laser systems in the storage and entanglement zones, a separation of $10\text{ }\mu\text{m}$ was maintained between the two regions.

B. BENCHMARKS

To comprehensively evaluate the performance and scalability of the ZAP compiler, we consider two categories of quantum circuits: structured algorithmic benchmarks and synthetic random circuits. The benchmark suite spans a broad range of system sizes, including small (2–10 qubits), medium (11–27 qubits), and large (28 or more qubits) instances.

1) STRUCTURED QUANTUM ALGORITHMS

We selected a diverse set of representative quantum algorithms to assess the compiler across different logical structures. The suite includes the following four classes of workloads.

- 1) Arithmetic and mathematical transforms, including Adder, Multiplier, and the Quantum Fourier Transform (QFT), which often exhibit dense, hierarchical entanglement patterns, such as the butterfly network in QFT.
- 2) Optimization and simulation circuits, including the Quantum Approximate Optimization Algorithm (QAOA), Ising models, and Max-Cut (SAT) solvers, which typically exhibit regular grid-like or near-neighbor connectivity.
- 3) Data-structure and machine-learning circuits, including quantum random access memory (QRAM), K-Nearest Neighbors (KNN), and variational quantum classifiers (VQC), with QRAM notably featuring a complex tree-like topology.
- 4) State-preparation circuits, including GHZ, W-state, and Cat-state circuits, which provide high-entanglement-depth benchmarks for fidelity scaling.

These workloads allow us to examine how different compiler heuristics navigate architecture-specific constraints, particularly the tradeoff between atom transport and crosstalk in irregular interaction patterns such as QRAM.

2) RANDOM CIRCUIT GENERATION

To evaluate scalability without bias toward any specific algorithmic motif, we generate synthetic random circuits from three-regular interaction graphs. For each

tested qubit count N , we use the NetworkX function `nx.random_regular_graph(d=3, n=N)` to construct an undirected graph $G = (V, E)$, where each vertex $v \in V$ represents a logical qubit and each edge $(u, v) \in E$ corresponds to one logical two-qubit CZ interaction between qubits u and v . Consequently, every qubit participates in exactly three entangling interactions, and each random instance contains $|E| = 3N/2$ two-qubit gates.

We choose three-regular graphs for two reasons. First, fixing the degree at $d = 3$ keeps the interaction density sparse and consistent across problem sizes, allowing the compiler to be evaluated under a controlled scaling regime rather than under varying gate densities. Second, random regular graphs suppress benchmark-specific structural artifacts, such as hubs, chains, and tree-like motifs, that could favor one heuristic over another. As a result, these circuits provide a clean average-case benchmark for studying how compilation time, execution time, movement overhead, and fidelity scale with the number of qubits. In this sense, the random-circuit study isolates scalability with respect to system size, whereas the structured benchmarks capture behavior on application-specific connectivity patterns.

C. BASELINE

We compare ZAP against three representative neutral-atom compilers: Enola, ZAC, and PowerMove. Enola targets nonzoned architectures and leverages graph-coloring-based scheduling to minimize circuit depth, thereby serving as a baseline that highlights the advantage of zoned architectures in suppressing crosstalk [21]. ZAC is a zoned neutral-atom compiler that employs iterative optimization (e.g., simulated annealing) for initial mapping and qubit reuse, and thus provides a baseline for evaluating the tradeoff between compilation quality and runtime [22]. PowerMove is a strong recent compiler that integrates multistage heuristics for joint scheduling and routing, making it a strong heuristic baseline for zoned architectures [23].

Since single-qubit gate implementations are identical across the compared compilers and their counts are not affected by compilation strategy, we exclude single-qubit gate fidelity from the evaluation metric to focus on the dominant sources of variation. Specifically, we evaluate fidelity in terms of two-qubit gates, atom transfer, decoherence, and crosstalk. Nevertheless, single-qubit gates are fully accounted for during compilation and scheduling and are included in the optimization process.

VII. EVALUATION RESULTS

We evaluate ZAP on structured benchmarks and random circuits in terms of fidelity, circuit execution time, and compilation time relative to prior neutral-atom compilers, and then use ablation studies to quantify the impact of its main design choices.

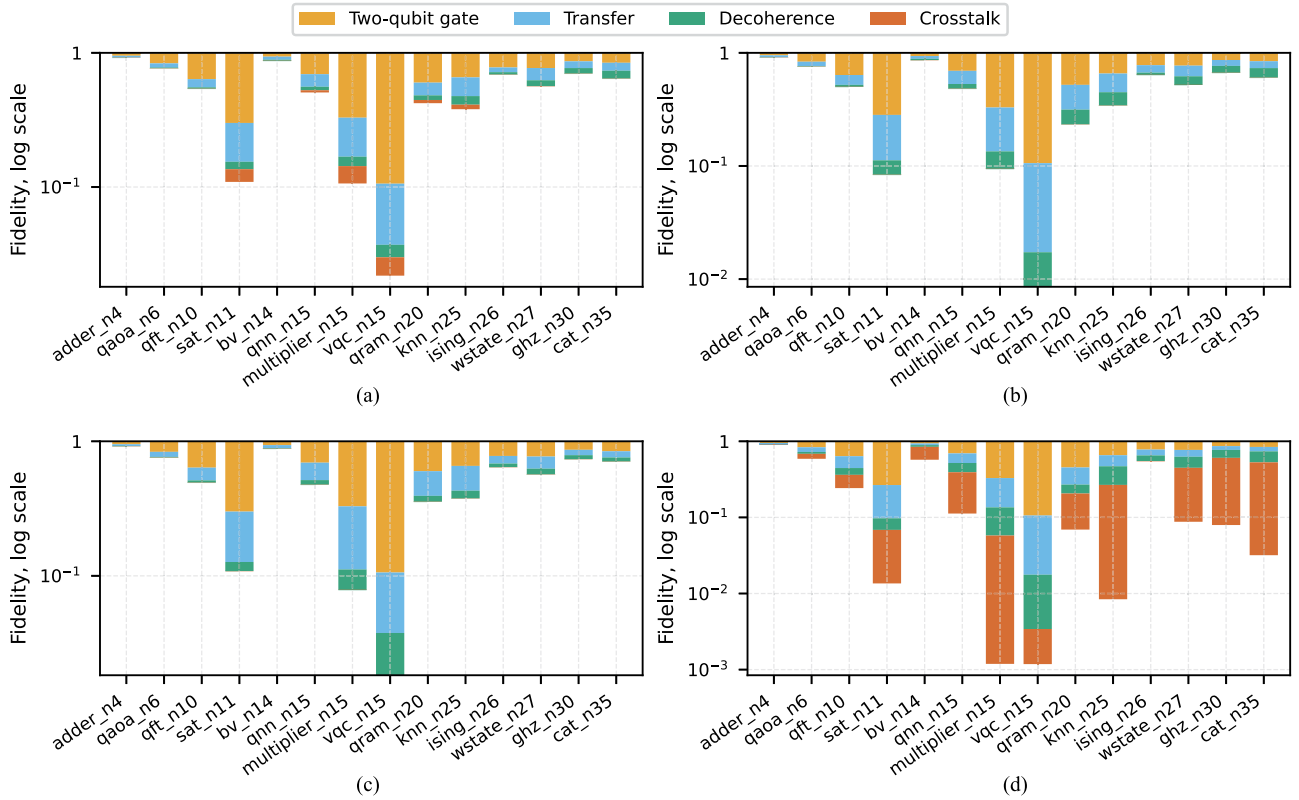


FIGURE 7. Structured-benchmark fidelity breakdown for four compilers. (a) ZAP, (b) ZAC, (c) PowerMove, and (d) Enola. Each stacked bar shows the remaining fidelity after accounting for two-qubit-gate error, atom-transfer error, decoherence, and crosstalk for one benchmark circuit. This figure highlights how different compilers trade off transport, decoherence, and crosstalk on the same workload suite.

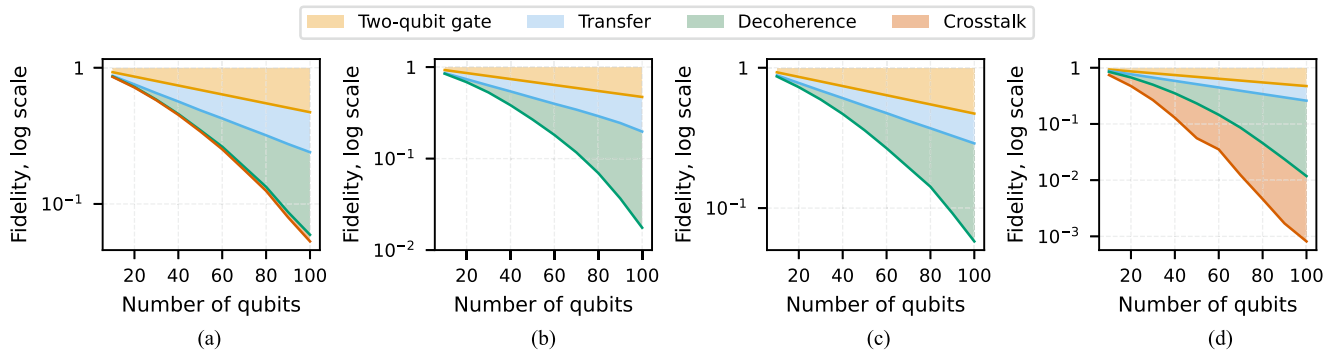


FIGURE 8. Random-circuit fidelity scaling on three-regular graph benchmarks for (a) ZAP, (b) ZAC, (c) PowerMove, and (d) Enola. Each benchmark is constructed by generating a random three-regular interaction graph and instantiating one CZ gate per edge. For each compiler, the colored regions show the cumulative fidelity degradation induced by two-qubit gates, atom transfer, decoherence, and crosstalk as the number of qubits increases from 10 to 100.

A. FIDELITY PERFORMANCE

To assess ZAP's fidelity performance, we estimate circuit fidelity under the hardware parameters described above on a benchmark suite spanning a range of circuit depths and problem sizes. Here, depth denotes the number of stages in the ASAP-separate schedule. The suite includes small (2 to 10 qubits), medium (11–27 qubits), and large (28 or more qubits) instances, enabling a broad evaluation of both performance and scalability.

We first examine the fidelity budget of the compiled circuits, with the structured-benchmark breakdown shown in Fig. 7 and the random-circuit scaling study shown in Fig. 8. These analyses include the main loss channels relevant to compilation: two-qubit-gate error, atom transfer between the SLM and AOD, decoherence, and crosstalk. Among these terms, the gate contribution is largely hardware-determined and therefore serves as a common background factor, whereas atom transfer, decoherence, and crosstalk

depend strongly on compilation decisions. Across both structured and random benchmarks, ZAP's zoned architecture and parallel transport strategy are particularly effective at suppressing crosstalk and controlling transfer overhead, while decoherence gradually becomes the dominant residual bottleneck as circuit size increases.

An important observation from Fig. 8 is that the fidelity curves of ZAP and PowerMove are much closer on random circuits than on structured algorithmic benchmarks. This behavior is expected. The random instances are generated from three-regular graphs, so for a given qubit count each compiler sees the same sparse interaction density and the same total number of two-qubit interactions ($3N/2$). Because these graphs are topologically homogeneous and lack high-contrast motifs such as the butterfly structure in QFT or the tree-like reuse pattern in QRAM, they provide fewer opportunities for a compiler to gain a large advantage from specialized scheduling or placement decisions. As a result, hardware-determined gate and decoherence terms dominate, while compiler-dependent differences in movement, reuse, and crosstalk tend to average out.

By contrast, structured algorithms amplify the effects of compiler design because they contain nonuniform connectivity patterns and repeated motifs that interact strongly with movement policies, qubit reuse, and zone transitions. In these workloads, differences in scheduling and placement quality accumulate systematically across stages, which is why the fidelity gaps in Fig. 7 are more pronounced than in the random-circuit study.

Our primary finding is that ZAP consistently outperforms the Enola baseline on both structured and random benchmarks. In the structured suite of Fig. 7, Enola exhibits the largest crosstalk-induced degradation, whereas ZAP preserves visibly higher fidelity across most workloads. The same qualitative trend appears in the random-circuit study of Fig. 8, although the gap is smaller because three-regular instances are more topologically homogeneous and therefore offer less room for compiler-dependent differentiation.

Relative to the stronger zoned baselines ZAC and PowerMove, ZAP's advantage is more workload-dependent but remains clear on the benchmarks that best expose compiler quality. As shown in Figs. 7 and 9, ZAP is typically best or near-best on structured workloads with irregular connectivity and nonuniform reuse, where scheduling, placement, and transport decisions accumulate over many stages. A representative example is qft_n10, where ZAP achieves a fidelity of 0.541 compared with 0.494 for PowerMove. By contrast, on more regular state-preparation circuits such as ghz_n30 and cat_n35, the gaps among zoned compilers are smaller because the movement pattern is simpler and leaves less room for compiler-level differentiation. This contrast reinforces the main point: ZAP's largest fidelity gains arise not from changing the hardware error model, but from making better tradeoffs among crosstalk, transport, and waiting time on structurally challenging workloads.

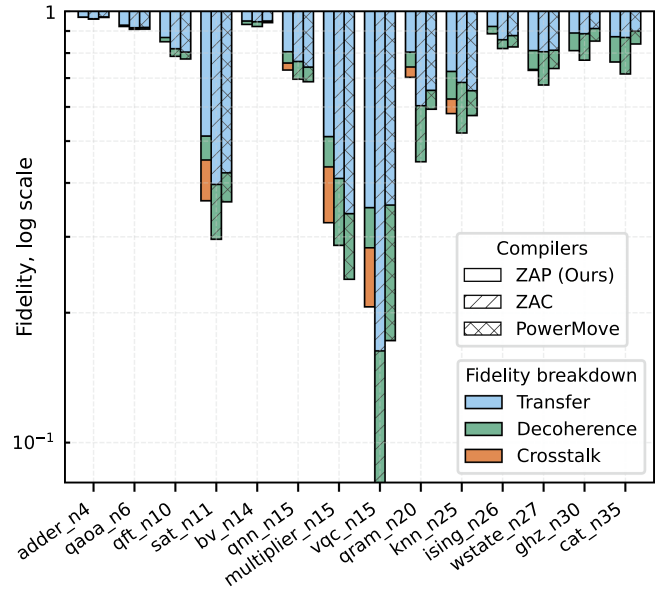


FIGURE 9. Direct comparison of compiler-dependent fidelity losses on the structured benchmark suite. For each benchmark, three adjacent bars correspond to ZAP, ZAC, and PowerMove, distinguished by hatch pattern. The stacked components show only the compiler-dependent loss channels—atom transfer, decoherence, and crosstalk—while gate-infidelity terms shared across compilers are omitted for clarity. This visualization isolates the source of ZAP's fidelity advantage over other zoned compilers.

A more fine-grained reading of the fidelity breakdown reveals that different benchmark families stress different compiler decisions. For state-preparation circuits, such as GHZ and Cat, the interaction pattern is highly regular and the reuse opportunities are easy to identify, so all zoned compilers exhibit relatively similar crosstalk behavior and the remaining differences mainly reflect movement and decoherence overhead. For arithmetic and transform circuits, such as Adder and QFT, long-range and repeated interaction patterns amplify the cost of unnecessary shuttling, making transfer and decoherence much more visible in the fidelity budget. For optimization and simulation benchmarks, such as Ising and QAOA, the interaction structure is more localized and regular, so the gaps among zoned compilers are moderate but still reflect how effectively each method controls movement accumulation over many stages. The clearest separation appears in irregular data-access and machine-learning-style workloads, such as QRAM and KNN, where nonuniform connectivity and qubit reuse patterns interact strongly with zone transitions. In those cases, ZAP benefits most from its cost-aware idle-qubit management and routing-aware placement, which reduce avoidable return-and-reload cycles and thereby lower both transport-related loss and movement-induced decoherence.

This interpretation is reinforced by the direct comparison of compiler-dependent error channels in Fig. 9. There, only transfer, decoherence, and crosstalk losses are shown, allowing the influence of compiler decisions to be isolated

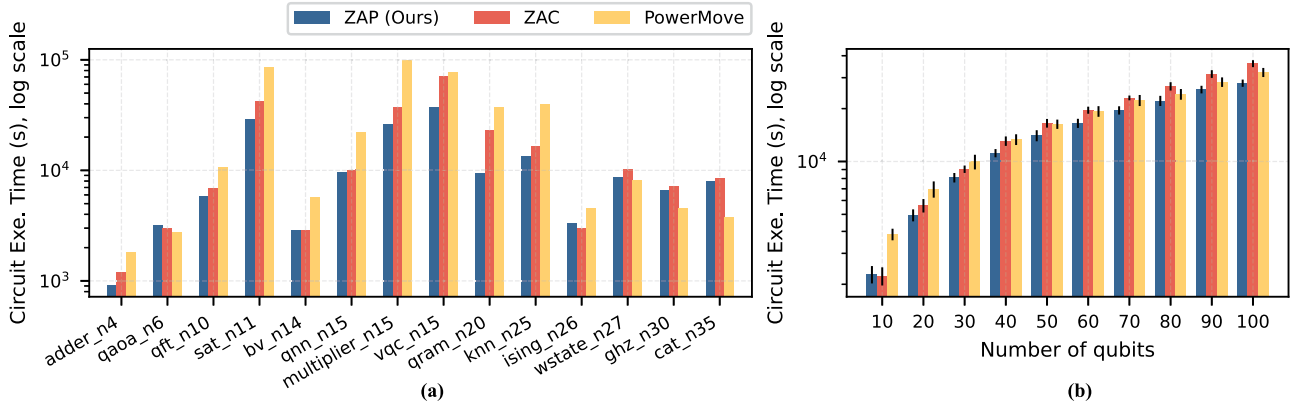


FIGURE 10. Circuit execution time comparison for zoned compilers. (a) Execution time on the structured benchmark suite. (b) Execution time versus qubit count for random three-regular circuits, with one CZ interaction instantiated per graph edge. We report the physical makespan of the compiled schedule, including gate durations, SLM–AOD transfers, in-array movement, and extra delays inserted to resolve AOD conflicts. Unlike compilation time, execution time is governed mainly by shared hardware primitives, so the gap among ZAP, ZAC, and PowerMove is more modest; nevertheless, ZAP remains consistently competitive and is often the fastest due to reduced shuttling overhead.

from gate-infidelity terms common to all methods. The figure shows that ZAP almost completely suppresses crosstalk on many benchmarks while keeping transfer and decoherence losses competitive with, and often slightly below, those of ZAC and PowerMove. Together with the random-circuit trends in Fig. 8, this indicates that ZAP’s fidelity advantage comes not from changing the underlying hardware error model, but from making better scheduling, placement, and routing decisions under the same hardware constraints.

While ZAP significantly reduces compilation-dependent errors, decoherence remains largely hardware-limited. Improvements in vacuum quality, magnetic-field stability, laser-noise suppression, and coherence-preserving control techniques can further extend the effective coherence time of neutral-atom qubits. As these hardware-level limitations are relaxed, the relative impact of compilation choices on crosstalk and transport overhead will become even more pronounced. By effectively managing these compilation-level errors, ZAP is well-positioned to maximize the potential of future-generation neutral-atom hardware.

B. CIRCUIT EXECUTION TIME

We next compare the physical execution time of the compiled circuits, shown in Fig. 10. For each compiled schedule, we report the hardware-level makespan, including two-qubit gate durations, SLM–AOD transfer time, in-array transport time, and the additional waiting time introduced by AOD row/column conflict resolution and path-clearing constraints. This metric is directly relevant to fidelity because a longer makespan increases the circuit’s exposure to decoherence. We restrict this comparison to zoned baselines, since ZAC, PowerMove, and ZAP share the same storage–entanglement execution model; Enola targets a different, nonzoned architecture, making a like-for-like physical-runtime comparison less meaningful.

The gap in Fig. 10 is naturally smaller than the gap in classical compilation time because ZAP, ZAC, and PowerMove

all operate under the same underlying hardware primitives. In other words, no compiler can arbitrarily change the native gate speed or the atom-transport velocity. A fair runtime comparison therefore asks a more focused question: given the same hardware parameters, which compiler introduces less unnecessary movement and less scheduling overhead in the final executable schedule?

Under this criterion, ZAP performs favorably. Across both structured and random benchmarks, ZAP remains consistently competitive and is often the fastest among the zoned compilers. The advantage is particularly clear on larger and more irregular workloads, where ZAP’s ASAP-separate scheduling, cost-aware idle-qubit management, and routing-aware placement reduce avoidable return-and-reload cycles and shorten the total transport path before each entangling stage. Consequently, ZAP improves fidelity by suppressing crosstalk without paying for that benefit with longer execution time.

C. CLASSICAL COMPILATION EFFICIENCY

1) COMPILATION SPEEDUP

Compilation time is a major system-level bottleneck in the quantum computing stack because it directly affects workflow throughput and the practicality of iterative methods. ZAP addresses this bottleneck through compiler–architecture co-design for a partitioned neutral-atom platform. In particular, ZAP uses a deterministic, single-pass compilation flow that maps logical qubits to physical atoms without the iterative search procedures used in ZAC and Enola and without the multistage co-optimization framework used in PowerMove.

We first compare ZAP with ZAC, PowerMove, and Enola on the structured benchmarks used in the fidelity study. As shown in Fig. 11(a), ZAP delivers a substantial performance advantage across the entire suite. Relative to ZAC and PowerMove, compilation time is reduced from tens of seconds to below 0.1 s on many benchmarks, corresponding

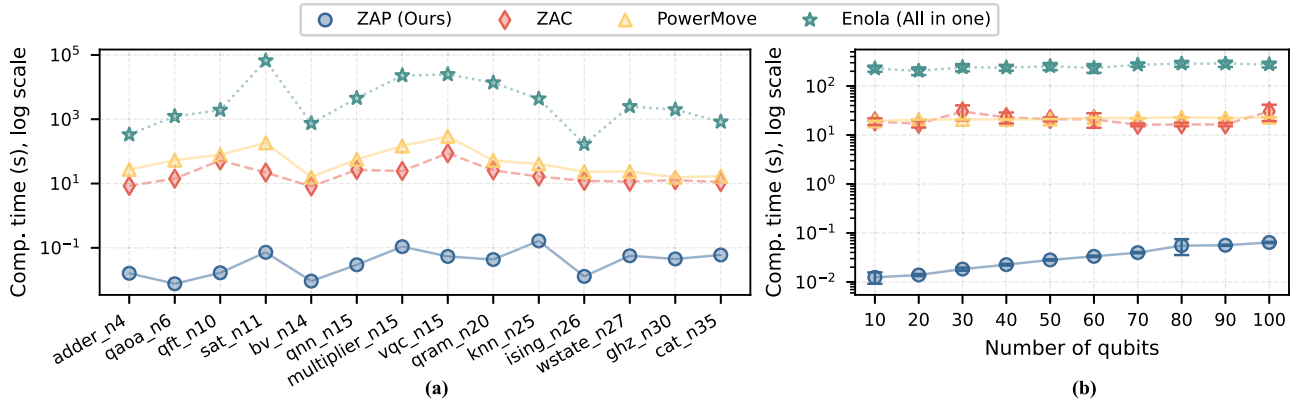


FIGURE 11. Compilation time comparison for ZAP, ZAC, Enola, and PowerMove. (a) Compilation time for the suite of structured quantum algorithms. (b) Compilation time as a function of qubit count for randomly generated circuits. ZAP exhibits substantially lower compilation time and better scaling.

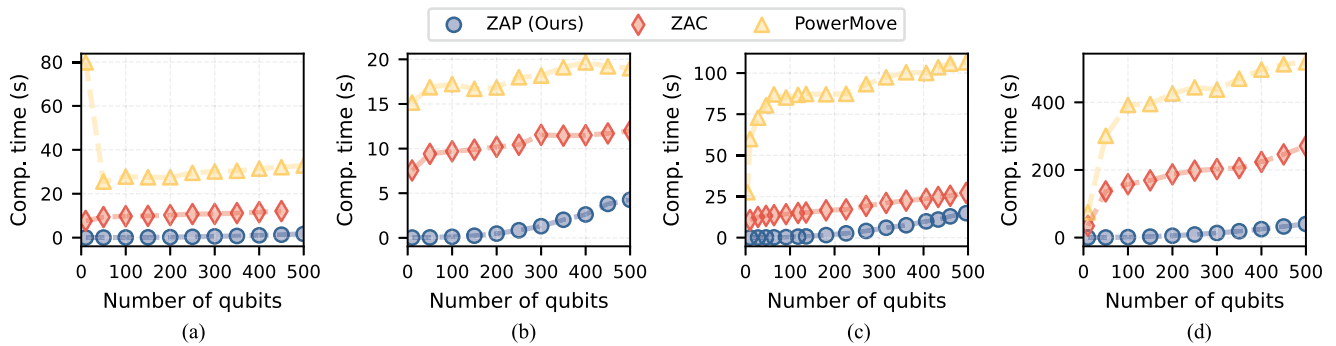


FIGURE 12. Compilation time scalability comparison between ZAP, ZAC, and PowerMove for four representative algorithms. From left to right, the panels correspond to (a) Ising, (b) Cat, (c) Adder, and (d) QFT. Each panel plots compilation time versus the number of qubits up to 500. Enola is omitted from this comparison due to its substantially larger compilation overhead, as established in Fig. 11. Across all four workloads, ZAP consistently exhibits both the lowest absolute compilation time and the flattest growth trend.

to consistent two- to three-order-of-magnitude speedups. The gap relative to Enola is even larger, reaching four to six orders of magnitude on this suite. These results show that ZAP largely removes compilation as a practical bottleneck and enables interactive or iterative compilation workflows.

To verify that this advantage is not specific to structured algorithms, we also evaluate randomly generated circuits of increasing size. Fig. 11(b) shows that ZAP remains below 0.1 s even up to 100 qubits, whereas ZAC and PowerMove scale into the tens of seconds. This corresponds to speedups of roughly $100\times$ to $1000\times$ and indicates that ZAP's efficiency stems from its underlying compilation strategy rather than from workload-specific structure.

Fast compilation is only useful if it also scales to large problem sizes. We therefore evaluate ZAP on four representative algorithms—Ising, Cat, Adder, and QFT—up to 500 qubits (see Fig. 12), omitting Enola because its compilation overhead is already substantially larger in Fig. 11. The results show that ZAP's performance advantage persists at scale. Even at 500 qubits, ZAP remains orders of magnitude faster than both ZAC and PowerMove. Notably, ZAP compiles a 500-qubit QFT circuit in under a minute,

whereas the iterative search procedures in ZAC and the heavier co-optimization flow in PowerMove become computationally prohibitive. This scalability is not merely an incremental improvement; it is a key enabler for exploring large-scale quantum algorithms that are difficult to study with iterative compilation methods.

2) COMPLEXITY ANALYSIS

The divergence in compilation scaling observed in Figs. 11 and 12 reflects the underlying algorithmic structure of each framework.

ZAC follows a staged heuristic pipeline rather than a single monolithic optimizer. Its runtime arises from several coupled subproblems, including gate scheduling, simulated-annealing-based initial placement, intermediate placement via minimum-weight bipartite matching, and routing under movement-conflict constraints using MIS-style selection. Consequently, its empirical complexity depends strongly on topology and geometry. When interaction graphs are locally bounded and structurally homogeneous, as in random three-regular instances, conflict patterns are more regular and scaling is typically smoother. For

structured circuits with long-range and highly nonuniform dependencies, such as QFT-like patterns, conflict graphs become denser and less regular, leading to larger runtime variability.

PowerMove also adopts a multistage heuristic flow, combining gate-stage formation, simulated-annealing-based initial placement, and movement scheduling with conflict-aware grouping. Relative to a simpler single-pass compiler, this introduces additional optimization stages and therefore higher runtime overhead. As the number of qubits and the interaction depth increase, the total optimization and scheduling workload grows accordingly.

In contrast, ZAP leverages the structural constraints of the zoned architecture to convert a potentially intractable search problem into a deterministic, single-pass process. By partitioning the array into storage and entanglement zones, ZAP reduces placement and routing to a sequence of greedy look-ahead decisions.

To make this statement more precise, let $N = |Q|$ be the number of qubits, $M = |S|$ the number of storage sites, $P = |\mathcal{E}|$ the number of entanglement sites, G the total number of two-qubit gates, and m_ℓ the number of qubit moves generated at stage ℓ . In the current implementation, the routing-aware initial mapping consists of: 1) distance preprocessing over storage and entanglement sites, which costs $O(MP)$; 2) priority-weight computation over the scheduled gates, which is linear in the schedule size; and 3) assignment of qubits to storage sites using the AOD-conflict-aware score. The last step is dominant: for each qubit, the algorithm scans the remaining candidate storage sites and compares the induced movement vector against the already planned vectors. In the worst case, with $M = \Theta(N)$ and a linear number of previously planned vectors, this yields $O(N^3)$ time for the initial mapping phase.

The stage-wise placement logic is cheaper. Idle-qubit management scales with the number of qubits currently resident in the entanglement zone and the cost of querying the next two-qubit reuse, while active placement scales with the number of two-qubit gates in the stage and the search for a free entanglement pair. Routing then constructs dependence and conflict relations among the moves in each stage and applies a greedy MIS routine, giving a cost of $\alpha \sum_\ell m_\ell^2$ in the worst case. Therefore, we do not claim that the full ZAP compiler is strictly linear in the most general asymptotic sense.

What our data support is a practical statement: ZAP is single-pass and noniterative, so it avoids the repeated global search loops used by ZAC, Enola, and PowerMove. Moreover, the zoned architecture bounds the number of simultaneously relevant entanglement sites and limits the routing batch width in each stage. Under these hardware-imposed constraints, the effective values of m_ℓ remain small relative to the total circuit size, so the observed end-to-end compilation time grows close to linearly over the regime we evaluate. These curves do not constitute a formal asymptotic proof that no future intersection can occur outside the evaluated range,

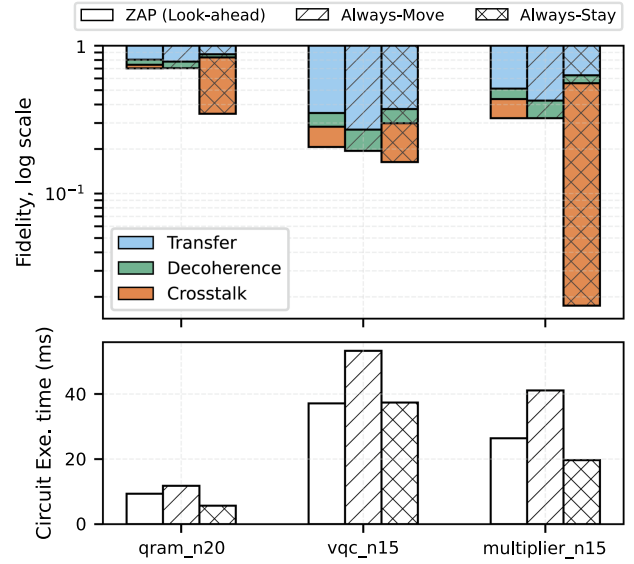


FIGURE 13. Ablation of the idle-qubit keep-versus-move policy, illustrating the crosstalk-versus-transport tradeoff in ZAP's placement strategy. The top panel reports fidelity breakdowns on qram_n20, vqc_n15, and multiplier_n15 for three variants: ZAP with the dynamic look-ahead rule, Always-Move, and Always-Stay. The bottom panel reports the corresponding circuit execution time. The dynamic policy captures most of the crosstalk suppression of always-move while avoiding much of its runtime overhead, and it avoids the severe crosstalk growth of Always-Stay on structured workloads.

but they do indicate that ZAP has a substantially smaller effective slope and better practical scalability than the baselines over the problem sizes of interest.

D. ABLATION STUDIES

1) STRATEGY ABLATION: LOOK-AHEAD TRADEOFF

The most informative ablation inside ZAP is the keep-versus-move decision for idle qubits in the entanglement zone. As described by the cost comparison above and Algorithm 2, ZAP explicitly compares the predicted crosstalk penalty of leaving a qubit in place against the transfer and decoherence penalty of returning it to storage. To isolate the effect of this rule, we compare the full dynamic policy against two static baselines: Always-Move, which aggressively returns every idle qubit to storage, and Always-Stay, which keeps idle qubits in the entanglement zone whenever possible. Fig. 13 shows that neither extreme is uniformly desirable. Always-Move suppresses crosstalk, but its larger number of transfers increases execution time and can reintroduce transfer and decoherence loss. Always-Stay minimizes movement, but it can accumulate substantial crosstalk on structured circuits with nonuniform reuse distances.

The dynamic look-ahead policy provides the best overall tradeoff. On qram_n20 and multiplier_n15, ZAP attains fidelity close to Always-Move while noticeably reducing circuit execution time, showing that it captures most of the crosstalk benefit without paying the full shuttling cost. On multiplier_n15, the gap relative to Always-Stay is especially dramatic: keeping idle qubits in place leads to a collapse

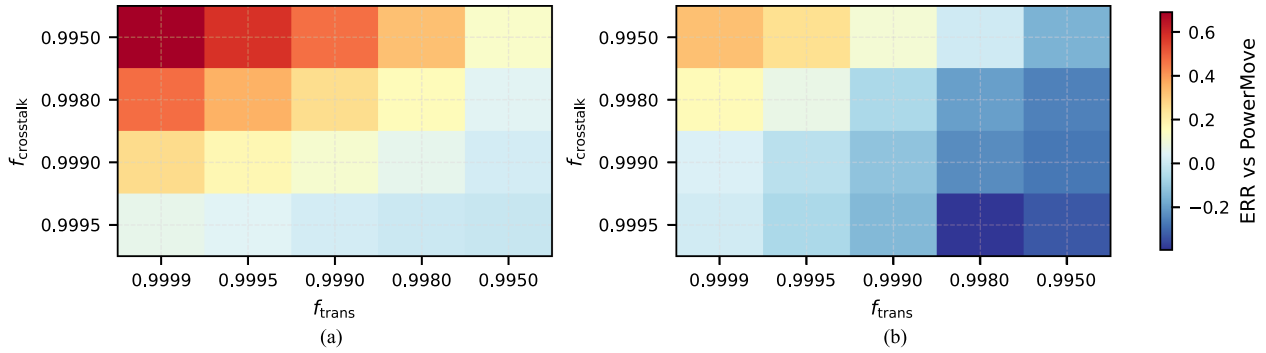


FIGURE 14. ERR of ZAP relative to PowerMove under different transfer and crosstalk fidelities. The heatmaps sweep f_{tr} and f_{xtalk} for (a) qram_n20 and (b) random_n50. Positive values indicate that ZAP achieves lower error than PowerMove. ZAP's advantage is strongest when crosstalk is costly and transfer is relatively reliable, and the effect is markedly stronger on the structured QRAM benchmark than on the topology-homogeneous random circuit.

in fidelity driven by crosstalk, whereas the dynamic policy selectively evacuates only the qubits for which the expected crosstalk cost exceeds the movement penalty. On vqc_n15, where the three policies are closer, ZAP still achieves the strongest balance between the top-panel fidelity breakdown and the bottom-panel runtime. These results confirm that the benefit of look-ahead is not simply lower movement or lower crosstalk in isolation, but the ability to choose between them on a stage-by-stage basis.

2) HARDWARE PARAMETER SENSITIVITY

To test whether the benefit of the dynamic policy is robust beyond a single hand-tuned hardware setting, we sweep the transfer fidelity f_{tr} and the crosstalk fidelity f_{xtalk} and measure the resulting error reduction rate (ERR) of ZAP relative to PowerMove

$$ERR = \frac{E_{PowerMove} - E_{ZAP}}{E_{PowerMove}} \quad (21)$$

where $E_{PowerMove}$ and E_{ZAP} denote the error rates of PowerMove and ZAP, respectively, with error rate defined as $E = 1 - F$ for a given fidelity F .

We use qram_n20 as a representative structured benchmark and random_n50 as a topology-homogeneous random circuit. As shown in Fig. 14(a), the response on qram_n20 is clear and interpretable rather than arbitrary: ZAP remains advantageous over a broad region of the parameter space, with the largest gains appearing when crosstalk is relatively severe and transport is relatively reliable. In that regime, the look-ahead policy can exploit the heterogeneous reuse structure of QRAM to suppress crosstalk without incurring unnecessary shuttling.

The random_n50 heatmap is more moderate, as expected, and does not mirror the structured case. Random three-regular circuits intentionally remove strong motifs, long-range reuse asymmetries, and other high-contrast features that make adaptive keep-versus-move decisions especially valuable. As a result, ZAP's relative advantage narrows and can become neutral or slightly negative in movement-dominated regimes where transfer loss is more important

than crosstalk. We view this not as a contradiction but as a refinement of the claim: ZAP's dynamic policy is most valuable on structured workloads with heterogeneous reuse patterns, whereas on topology-homogeneous random circuits it remains competitive but has less structure to exploit. This interpretation is consistent with the earlier fidelity results, where the gaps among compilers are markedly larger on structured benchmarks than on random three-regular circuits.

VIII. RELATED WORK

Compilation strategies must be tailored to specific quantum hardware due to differences in native gates and chip topologies [31]. For superconducting platforms, methods, such as the SABRE algorithm [32], randomized circuit compilation [33], and AI-based techniques [34], have been developed, and one-shot global optimization approaches, such as ILP [35], MaxSAT [36], and subgraph isomorphism [37], have been used to solve placement and routing problems on superconducting platforms with fixed, local qubit connectivity. As Ion-trap and neutral-atom systems increasingly adopt distributed and heterogeneous architectures to improve scalability and connectivity, recent frameworks have begun to address the associated compilation and communication challenges [38], [39]. Resource-virtualized and hardware-aware compilation systems, such as QSteed, further improve real-device compilation and task management for practical quantum computing backends [40]. Strategies leveraging ion shuttling—such as TILT for linear traps [41], [42] and various methods for quantum charge-coupled device (QCCD) architectures [43], [44], [45]—primarily focus on efficient ion transport and mitigating performance drops as qubit numbers increase.

For neutral atom platforms, parallel qubit movement and global laser operations enable high connectivity and parallelism, which have inspired tailored compilation schemes. Early works, such as OLSQ-DPQA [46], focus on optimal layout synthesis using SMT solvers. Enola [21] further improves scheduling through graph edge coloring, reducing stages by $3.7\times$ and improving fidelity by up to $5.9\times$ over prior methods. Other works [47] study atom loss errors in

flexible atom arrays with long-distance interactions. A representative line of prior work focuses on nonzoned neutral-atom architectures with flexible qubit arrays and global addressing constraints, including Geyser [48], FPQA-C [49], Q-Pilot [50], Atomique [51], Qompose [52], and Enola [21]. In parallel, a distinct line of later work studies zoned neutral-atom architectures that explicitly separate storage and entanglement regions, including NALAC [53], Arctic [54], ZAC [22], and PowerMove [23]. These zoned approaches improve scalability and mitigate crosstalk through structured atom movement, scheduling, and reuse management. More recent follow-up works further integrate routing-aware and AOD-aware placement into zoned compilation, including the routing-aware placement method of Stade et al. [55] and the later scalable search framework of Stade et al. [56].

Collectively, these frameworks address the compilation and scheduling challenges of reconfigurable neutral atom architectures, enhancing scalability, fidelity, and efficiency. Our work primarily compares with Enola [21], which requires gate commutativity, ZAC [22], a reuse-aware compiler with iterative placement loops, and PowerMove [23], a strong recent compiler that utilizes a multistage heuristic framework to co-optimize gate scheduling and continuous atom routing. In contrast to all three approaches, our partitioned architecture and deterministic single-pass compiler minimize parallel atomic movement, thereby improving fidelity and substantially reducing compilation time without iterative overhead, offering a more robust and scalable approach to neutral atom quantum computing.

IX. DISCUSSION AND CONCLUSION

In this work, we presented ZAP, a co-designed zoned architecture and deterministic compiler for neutral-atom quantum computing. ZAP combines a hardware-structured storage–entanglement layout with ASAP-separate scheduling, routing-aware placement, cost-aware idle-qubit management, and conflict-aware routing. Rather than relying on repeated global search, the compiler resolves scheduling, placement, and transport in a single-pass flow that is explicitly matched to the constraints of AOD-based atom movement. The main contribution is therefore not an isolated heuristic, but the overall co-design between zoned hardware structure and a noniterative compilation strategy.

Our evaluation on structured benchmarks and random three-regular circuits shows that this co-design yields substantial practical benefits. Relative to the zoned baselines ZAC and PowerMove, ZAP typically reduces compilation time from tens of seconds to below 0.1 s and delivers speedups exceeding $1000\times$, while remaining competitive or superior in execution quality. Relative to the nonzoned Enola baseline, the compilation speedup exceeds $10\,000\times$ on the evaluated suite, while fidelity improves markedly because zoning strongly suppresses crosstalk. Among zoned compilers, ZAP's fidelity advantage is most pronounced on structured workloads with irregular connectivity and

nonuniform qubit reuse, where better scheduling, placement, and transport decisions accumulate over many stages; on random circuits, the fidelity gap is smaller but ZAP remains competitive and preserves the same scalability advantage. Importantly, these fidelity improvements do not come at the cost of longer physical runtime: across zoned baselines, ZAP is consistently competitive in execution time and is often the fastest because it reduces unnecessary shuttling and waiting overhead.

We also emphasize a practical systems perspective. For a one-off NISQ experiment with only tens to a few hundred qubits, reducing classical compilation time from minutes to milliseconds is not always the dominant end-to-end bottleneck compared with quantum runtime, hardware availability, and experimental overhead. In standard workflows, compilation is typically performed before atom loading and execution, so its main benefit is often improved experimental throughput rather than directly extending the coherence budget of a single shot. Even so, fast compilation is highly valuable in iterative and closed-loop settings, such as repeated benchmarking, calibration-aware recompilation, parameter sweeps, and adaptive workflows in which many related circuit instances must be regenerated under changing hardware conditions. In such scenarios, search-based latency can accumulate into a practical control-loop bottleneck. This systems consideration becomes even stronger in fault-tolerant settings, where the compiled objects shift from physical qubits to logical qubits or logical patches and repeated scheduling, routing, and resource-allocation decisions can themselves become a throughput limiter. It is also important from the perspective of quantum cloud services: because compilation and transpilation lie on the user-facing path before job submission, long compile latency degrades interactive feedback and makes the service less friendly to online users. This practical concern is reflected in existing cloud ecosystems, such as IBM Qiskit and the Quafu platform, which already expose transpilation or compilation as part of the cloud workflow [57], [58]. Although neutral-atom hardware has not yet been widely deployed as a public cloud service, the same requirement for low user-visible compilation latency will become increasingly important once such platforms are opened to online access.

More broadly, ZAP supports the view that hardware-structured, noniterative compilation is a promising path toward scalable and noise-aware neutral-atom computing. As hardware coherence and array size continue to improve, the relative importance of compiler decisions about crosstalk, transport, and waiting time will only increase. Future work includes exploring the co-design space of storage and entanglement zones, further reducing atom-transfer overhead, and extending the same philosophy toward fault-tolerant compilation. In that longer-term setting, the compilation objects would need to be lifted from physical atoms to logical qubits or logical patches, and the cost model would need to shift from physical movement and decoherence to logical

communication, distillation resources, and code-cycle latency. Although such an extension remains future work, the central principle of replacing repeated global search with hardware-aware deterministic compilation remains applicable.

ACKNOWLEDGMENT

The authors to thank Chenghong Zhu, Xudan Chai, and Yanwu Gu for their invaluable discussions.

REFERENCES

- [1] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, 2018, Art. no. 79, doi: [10.22331/q-2018-08-06-79](https://doi.org/10.22331/q-2018-08-06-79).
- [2] A. Montanaro, "Quantum algorithms: An overview," *npj Quantum Inf.*, vol. 2, no. 1, 2016, Art. no. 15023, doi: [10.1038/npjqi.2015.23](https://doi.org/10.1038/npjqi.2015.23).
- [3] C. D. Bruzewicz, J. Chiaverini, R. McConnell, and J. M. Sage, "Trapped-Ion quantum computing: Progress and challenges," *Appl. Phys. Rev.*, vol. 6, no. 2, 2019, Art. no. 021314, doi: [10.1063/1.5088164](https://doi.org/10.1063/1.5088164).
- [4] A. W. Harrow, A. Hassidim, and S. Lloyd, "Quantum algorithm for linear systems of equations," *Phys. Rev. Lett.*, vol. 103, no. 15, 2009, Art. no. 150502, doi: [10.1103/PhysRevLett.103.150502](https://doi.org/10.1103/PhysRevLett.103.150502).
- [5] M. Kjaergaard et al., "Superconducting qubits: Current state of play," *Annu. Rev. Condens. Matter Phys.*, vol. 11, no. 1, pp. 369–395, 2020, doi: [10.1146/annurev-conmatphys-031119-050605](https://doi.org/10.1146/annurev-conmatphys-031119-050605).
- [6] B. Lekitsch et al., "Blueprint for a microwave trapped Ion quantum computer," *Sci. Adv.*, vol. 3, no. 2, 2017, Art. no. e1601540, doi: [10.1126/sciadv.1601540](https://doi.org/10.1126/sciadv.1601540).
- [7] S. McArdle, S. Endo, A. Aspuru-Guzik, S. C. Benjamin, and X. Yuan, "Quantum computational chemistry," *Rev. Modern Phys.*, vol. 92, no. 1, 2020, Art. no. 015003, doi: [10.1103/RevModPhys.92.015003](https://doi.org/10.1103/RevModPhys.92.015003).
- [8] Y. Cao et al., "Quantum chemistry in the age of quantum computing," *Chem. Rev.*, vol. 119, no. 19, pp. 10856–10915, 2019, doi: [10.1021/acs.chemrev.8b00803](https://doi.org/10.1021/acs.chemrev.8b00803).
- [9] J.-P. Aumasson, "The impact of quantum computing on cryptography," *Comput. Fraud Secur.*, vol. 2017, no. 6, pp. 8–11, 2017, doi: [10.1016/S1361-3723\(17\)30051-9](https://doi.org/10.1016/S1361-3723(17)30051-9).
- [10] T. M. Fernandez-Carames and P. Fraga-Lamas, "Towards post-quantum blockchain: A review on blockchain cryptography resistant to quantum computing attacks," *IEEE Access*, vol. 8, pp. 21091–21116, 2020, doi: [10.1109/ACCESS.2020.2968985](https://doi.org/10.1109/ACCESS.2020.2968985).
- [11] J. Choi and J. Kim, "A tutorial on quantum approximate optimization algorithm (QAOA): Fundamentals and applications," in *Proc. 2019 Int. Conf. Inf. Commun. Technol. Convergence*, 2019, pp. 138–142, doi: [10.1109/ICTC46691.2019.8939749](https://doi.org/10.1109/ICTC46691.2019.8939749).
- [12] L. Henriot et al., "Quantum computing with neutral atoms," *Quantum*, vol. 4, 2020, Art. no. 327, doi: [10.22331/q-2020-09-21-327](https://doi.org/10.22331/q-2020-09-21-327).
- [13] K. Wintersperger et al., "Neutral atom quantum computing hardware: Performance and end-user perspective," *EPJ Quantum Technol.*, vol. 10, no. 1, 2023, Art. no. 32, doi: [10.1140/epjqt/s40507-023-00190-1](https://doi.org/10.1140/epjqt/s40507-023-00190-1).
- [14] I. Cong, H. Levine, A. Keesling, D. Bluvstein, S.-T. Wang, and M. D. Lukin, "Hardware-efficient, fault-tolerant quantum computation with Rydberg atoms," *Phys. Rev. X*, vol. 12, no. 2, 2022, Art. no. 021049, doi: [10.1103/PhysRevX.12.021049](https://doi.org/10.1103/PhysRevX.12.021049).
- [15] T. Graham et al., "Multi-qubit entanglement and algorithms on a neutral-atom quantum computer," *Nature*, vol. 604, no. 7906, pp. 457–462, 2022, doi: [10.1038/s41586-022-04603-6](https://doi.org/10.1038/s41586-022-04603-6).
- [16] D. Bluvstein et al., "Logical quantum processor based on reconfigurable atom arrays," *Nature*, vol. 7997, pp. 58–65, 2024, doi: [10.1038/s41586-023-06927-3](https://doi.org/10.1038/s41586-023-06927-3).
- [17] H. J. Manetsch, G. Nomura, E. Bataille, X. Lv, K. H. Leung, and M. Endres, "A tweezer array with 6,100 highly coherent atomic qubits," *Nature*, vol. 647, pp. 60–67, 2025, doi: [10.1038/s41586-025-09641-4](https://doi.org/10.1038/s41586-025-09641-4).
- [18] S. J. Evered et al., "High-fidelity parallel entangling gates on a neutral-atom quantum computer," *Nature*, vol. 622, no. 7982, pp. 268–272, Oct. 2023, doi: [10.1038/s41586-023-06481-y](https://doi.org/10.1038/s41586-023-06481-y).
- [19] D. Bluvstein et al., "A quantum processor based on coherent transport of entangled atom arrays," *Nature*, vol. 604, no. 7906, pp. 451–456, 2022, doi: [10.1038/s41586-022-04592-6](https://doi.org/10.1038/s41586-022-04592-6).
- [20] P. Murali, N. M. Linke, M. Martonosi, A. J. Abhari, N. H. Nguyen, and C. H. Alderete, "Full-stack, real-system quantum computer studies: Architectural comparisons and design insights," in *Proc. 46th Int. Symp. Comput. Archit.*, 2019, pp. 527–540, doi: [10.1145/3307650.3322273](https://doi.org/10.1145/3307650.3322273).
- [21] D. B. Tan, W.-H. Lin, and J. Cong, "Compilation for dynamically field-programmable Qubit arrays with efficient and provably near-optimal scheduling," 2024, *arXiv:2405.15095*, doi: [10.1145/3658617.3697778](https://doi.org/10.1145/3658617.3697778).
- [22] W.-H. Lin, D. B. Tan, and J. Cong, "Reuse-aware compilation for zoned quantum architectures based on neutral atoms," in *Proc. 2025 IEEE Int. Symp. High Perform. Comput. Archit.*, 2025, pp. 127–142, doi: [10.1109/HPCA61900.2025.00021](https://doi.org/10.1109/HPCA61900.2025.00021).
- [23] J. Ruan, X. Fang, H. Zhang, A. Li, T. Humble, and Y. Ding, "PowerMove: Optimizing compilation for neutral atom quantum computers with zoned architecture," in *Proc. 30th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2025, pp. 163–178, doi: [10.1145/3676642.3736128](https://doi.org/10.1145/3676642.3736128).
- [24] C. M. Dawson and M. A. Nielsen, "The Solovay-Kitaev algorithm," 2005, *arXiv:quant-ph/0505030*, doi: [10.48550/arXiv.quant-ph/0505030](https://doi.org/10.48550/arXiv.quant-ph/0505030).
- [25] H. Levine et al., "High-fidelity control and entanglement of Rydberg-atom qubits," *Phys. Rev. Lett.*, vol. 121, Sep. 2018, Art. no. 123603, doi: [10.1103/PhysRevLett.121.123603](https://doi.org/10.1103/PhysRevLett.121.123603).
- [26] M. Saffman, T. G. Walker, and K. Mølmer, "Quantum information with Rydberg atoms," *Rev. Mod. Phys.*, vol. 82, pp. 2313–2363, Aug. 2010, doi: [10.1103/RevModPhys.82.2313](https://doi.org/10.1103/RevModPhys.82.2313).
- [27] D. Bluvstein et al., "A fault-tolerant neutral-atom architecture for universal quantum computation," *Nature*, vol. 649, pp. 39–46, 2026, doi: [10.1038/s41586-025-09848-5](https://doi.org/10.1038/s41586-025-09848-5).
- [28] D. Jaksch, J. I. Cirac, P. Zoller, S. L. Rolston, R. Côté, and M. D. Lukin, "Fast quantum gates for neutral atoms," *Phys. Rev. Lett.*, vol. 85, pp. 2208–2211, Sep. 2000, doi: [10.1103/PhysRevLett.85.2208](https://doi.org/10.1103/PhysRevLett.85.2208).
- [29] M. D. Lukin et al., "Dipole blockade and quantum information processing in mesoscopic atomic ensembles," *Phys. Rev. Lett.*, vol. 87, no. 3, Jun. 2001, Art. no. 037901, doi: [10.1103/PhysRevLett.87.037901](https://doi.org/10.1103/PhysRevLett.87.037901).
- [30] A. Gaëtan et al., "Observation of collective excitation of two individual atoms in the Rydberg blockade regime," *Nature Phys.*, vol. 5, no. 2, pp. 115–118, Jan. 2009, doi: [10.1038/nphys1183](https://doi.org/10.1038/nphys1183).
- [31] C. Zhu et al., "Quantum compiler design for qubit mapping and routing: A cross-architectural survey of superconducting, trapped-ion, and neutral atom systems," 2025, *arXiv:2505.16891*, doi: [10.48550/arXiv.2505.16891](https://doi.org/10.48550/arXiv.2505.16891).
- [32] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2019, pp. 1001–1014, doi: [10.1145/3297858.3304023](https://doi.org/10.1145/3297858.3304023).
- [33] A. Hashim et al., "Randomized compiling for scalable quantum computing on a noisy superconducting quantum processor," 2020, *arXiv:2010.00215*, doi: [10.1103/PhysRevX.11.041039](https://doi.org/10.1103/PhysRevX.11.041039).
- [34] Z. Wang et al., "Quantum compiling with reinforcement learning on a superconducting processor," 2024, *arXiv:2406.12195*, doi: [10.48550/arXiv.2406.12195](https://doi.org/10.48550/arXiv.2406.12195).
- [35] R. Wille, L. Burgholzer, and A. Zulehner, "Mapping quantum circuits to IBM QX architectures using the minimal number of SWAP and H operations," in *Proc. 56th Annu. Des. Automat. Conf. 2019*, 2019, Art. no. 142, doi: [10.1145/3316781.3317859](https://doi.org/10.1145/3316781.3317859).
- [36] A. Molavi, A. Xu, M. Diges, L. Pick, S. Tannu, and A. Albarghouthi, "Qubit mapping and routing via MaxSAT," in *Proc. 55th IEEE/ACM Int. Symp. Microarchit.*, 2022, pp. 1078–1091, doi: [10.1109/MICRO56248.2022.00077](https://doi.org/10.1109/MICRO56248.2022.00077).
- [37] S. Li, X. Zhou, and Y. Feng, "Qubit mapping based on subgraph isomorphism and filtered depth-limited search," *IEEE Trans. Comput.*, vol. 70, no. 11, pp. 1777–1788, Nov. 2021, doi: [10.1109/TC.2020.3023247](https://doi.org/10.1109/TC.2020.3023247).
- [38] D. Xiang et al., "AdaptDQC: Adaptive distributed quantum computing with quantitative performance analysis," *IEEE Trans. Comput.*, vol. 74, no. 10, pp. 3277–3290, Oct. 2025, doi: [10.1109/TC.2025.3586027](https://doi.org/10.1109/TC.2025.3586027).
- [39] C. Tao et al., "Qtenon: Towards low-latency architecture integration for accelerating hybrid quantum-classical computing," in *Proc. 52nd Annu. Int. Symp. Comput. Archit.*, 2025, pp. 299–312, doi: [10.1145/3695053.3731087](https://doi.org/10.1145/3695053.3731087).
- [40] H.-Z. Xu et al., "A resource-virtualized and hardware-aware quantum compilation framework for real quantum computing processors," *Research*, vol. 8, 2025, Art. no. 0947, doi: [10.34133/research.0947](https://doi.org/10.34133/research.0947).

- [41] X.-C. Wu et al., "TILT: Achieving higher fidelity on a trapped-ion linear-tape quantum computing architecture," in *Proc. 2021 IEEE Int. Symp. High- Perform. Comput. Archit.*, 2021, pp. 153–166, doi: [10.1109/HPCA51647.2021.00023](https://doi.org/10.1109/HPCA51647.2021.00023).
- [42] X. Wu, C. Zhu, J. Wang, and X. Wang, "BOSS: Blocking algorithm for optimizing shuttling scheduling in ion Trap," in *Proc. 2025 IEEE Int. Symp. High Perform. Comput. Architecture*, 2025, pp. 290–303, doi: [10.1109/HPCA61900.2025.00032](https://doi.org/10.1109/HPCA61900.2025.00032).
- [43] K. E. Stevens, J. M. Amini, S. C. Doret, G. Mohler, C. Volin, and A. W. Harter, "Automating quantum experiment control: From circuit compilation to ion routing," *Quantum Inf. Process.*, vol. 16, pp. 1–20, 2017, doi: [10.1007/s11128-016-1454-1](https://doi.org/10.1007/s11128-016-1454-1).
- [44] X.-C. Wu et al., "ILP-based scheduling for linear-tape model trapped-ion quantum computers," in *Proc. Int. Conf. High Perform. Computing, Networking, Storage, Anal.*, Denver Co, 2019. [Online]. Available: https://sc19.supercomputing.org/proceedings/tech_poster/poster_files/rpost216s2-file3.pdf
- [45] C. Zhu, X. Wu, J. Wang, and X. Wang, "S-SYNC: Shuttle and swap co-optimization in quantum charge-coupled devices," in *Proc. 52nd Annu. Int. Symp. Comput. Archit.*, 2025, pp. 271–284, doi: [10.1145/3695053.3731084](https://doi.org/10.1145/3695053.3731084).
- [46] D. B. Tan, D. Bluvstein, M. D. Lukin, and J. Cong, "Compiling quantum circuits for dynamically field-programmable neutral atoms array processors," *Quantum*, vol. 8, 2024, Art. no. 1281, doi: [10.22331/q-2024-03-14-1281](https://doi.org/10.22331/q-2024-03-14-1281).
- [47] J. M. Baker, A. Litteken, C. Duckering, H. Hoffmann, H. Bernien, and F. T. Chong, "Exploiting long-distance interactions and tolerating atom loss in neutral atom quantum architectures," in *Proc. 2021 ACM/IEEE 48th Annu. Int. Symp. Comput. Archit.*, 2021, pp. 818–831, doi: [10.1109/ISCA52012.2021.00069](https://doi.org/10.1109/ISCA52012.2021.00069).
- [48] T. Patel, D. Silver, and D. Tiwari, "Geyser: A compilation framework for quantum computing with neutral atoms," in *Proc. 49th Annu. Int. Symp. Comput. Archit.*, 2022, pp. 383–395, doi: [10.1145/3470496.3527428](https://doi.org/10.1145/3470496.3527428).
- [49] H. Wang et al., "FPQA-C: A compilation framework for field programmable qubit array," 2023, *arXiv:2311.15123*, doi: [10.1109/ISCA59077.2024.00030](https://doi.org/10.1109/ISCA59077.2024.00030).
- [50] H. Wang et al., "Q-pilot: Field programmable quantum array compilation with flying ancillas," 2023, *arXiv:2311.16190*, doi: [10.1145/3649329.3658470](https://doi.org/10.1145/3649329.3658470).
- [51] H. Wang et al., "Atomique: A quantum compiler for reconfigurable neutral atom arrays," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit.*, IEEE, 2024, pp. 293–309, *arXiv:2311.16190v2*, doi: [10.1109/ISCA59077.2024.00030](https://doi.org/10.1109/ISCA59077.2024.00030).
- [52] D. Silver, T. Patel, and D. Tiwari, "Qompose: A technique to select optimal algorithm-specific layout for neutral atom quantum architectures," 2024, *arXiv:2409.19820*, doi: [10.48550/arXiv.2409.19820](https://doi.org/10.48550/arXiv.2409.19820).
- [53] Y. Stade, L. Schmid, L. Burgholzer, and R. Wille, "An abstract model and efficient routing for logical entangling gates on zoned neutral atom architectures," in *Proc. 2024 IEEE Int. Conf. Quantum Comput. Eng.*, 2024, pp. 784–795, doi: [10.1109/QCE60285.2024.00098](https://doi.org/10.1109/QCE60285.2024.00098).
- [54] E. Decker, "Arctic: A field programmable quantum array scheduling technique," 2024, *arXiv:2405.06183*, doi: [10.48550/arXiv.2405.06183](https://doi.org/10.48550/arXiv.2405.06183).
- [55] Y. Stade, W.-H. Lin, J. Cong, and R. Wille, "Routing-aware placement for zoned neutral atom-based quantum computing," in *Proc. 2025 IEEE/ACM Int. Conf. Comput. Aided Des.*, 2025, pp. 1–9, doi: [10.1109/ICCAD66269.2025.11240721](https://doi.org/10.1109/ICCAD66269.2025.11240721).
- [56] Y. Stade, L. Burgholzer, and R. Wille, "Search smarter, not harder: A scalable, high-quality zoned neutral atom compiler," 2025, *arXiv:2512.13790*, doi: [10.48550/arXiv.2512.13790](https://doi.org/10.48550/arXiv.2512.13790).
- [57] A. Javadi-Abhari et al., "Quantum computing with qiskit," 2024, *arXiv:2405.08810*, doi: [10.48550/arXiv.2405.08810](https://doi.org/10.48550/arXiv.2405.08810).
- [58] BAQIS Quafu Team, "PyQuafu documentation," 2024. [Online]. Available: <https://scq-cloud.github.io/0.4.0/index.html>